Seminario de Práctica de Informática

Trabajo Práctico N°3

Módulo 3

Código: INF275-11987

Comisión de la materia: CAT A - EDH

Alumno: Valentino Belli

DNI: 46497315

Legajo: VINF016781

Titular experto: Pablo Alejandro Virgolini

Titular disciplinar: Pablo Alejandro Virgolini

Fecha de entrega: 19/10/2026

Índice

Enunciado ... 3
Consignas ... 4
Formato Entregable ... 6
Correcciones TP2 ... 7
1- Etapa de análisis ... 7
2- Etapa de diseño ... 21
3- Etapa de implementación ... 21
4- Etapa de pruebas ... 23
5- Prototipo de interfaz ... 48
6- Definición de base de datos para el sistema ... 49
7- Diagrama entidad-relación de la base de datos ... 50
8- Creación de las tablas MySQL ... 51
9- Inserción, consulta y borrado de registro. ... 55
10- Definiciones de comunicación ... 61
Desarrollo TP3 ... 64
1. Tecnologías y herramientas ... 64
2. Decisiones tomadas durante el desarrollo ... 66
3. Organización y funcionamiento del código ... 68
4. Funcionamiento general de la aplicación ... 69
5. Requisitos para ejecutar el sistema ... 70
6. Pasos para preparar y ejecutar el proyecto ... 70
7. Conclusión ... 71

Enunciado

Los sistemas informáticos desempeñan un papel muy importante para la optimización de procesos en diversas áreas de cualquier organización. La tecnología brinda la posibilidad de automatizar tareas, recopilar y analizar datos a gran escala, agilizar procesos y proporcionar soluciones a desafíos complejos.

Te propongo que retomes lo que definiste en la actividad anterior y realices una entrega de acuerdo con lo solicitado en la consigna. Recuerda que puedes definir cualquier organización real y explorar oportunidades en relación con su seguridad, logística, gestión de inventarios, optimización de procesos industriales, análisis de datos, cuidado de la salud, la educación, o cualquier otra área.

Para continuar con la realización de un proyecto de desarrollo informático, se requiere aplicar los fundamentos de la programación orientada a objetos y sus pilares utilizando el lenguaje de programación Java. Es importante conocer las características esenciales del lenguaje tales como tipos de datos, estructuras de control y el manejo de excepciones.

Para la codificación, se deben utilizar estructuras condicionales y de repetición, para controlar el flujo de ejecución de un programa. Adicionalmente, se debe contemplar la declaración y creación de objetos en Java, así como el uso de constructores.

Consignas

Considera que continúas trabajando en la organización cuyo problema definiste resolver con un proyecto informático, y estás en el equipo de desarrollo que va a llevar adelante el mismo.

Recuerda que el proyecto informático definido debe ser distinto al considerado en la lectura y que el entregable final debe cumplir con los siguientes objetivos:

- Realizar el análisis del modelo de negocios para definir y justificar el proyecto.
- Aplicar el proceso unificado de desarrollo (PUD) para garantizar la calidad, escalabilidad y eficiencia en el ciclo de desarrollo.
- Utilizar una base de datos MySQL para la persistencia de los datos.
- Emplear Java como lenguaje de programación para el desarrollo del sistema.

A los fines del trabajo, para la implementación de la base de datos y el desarrollo con Java, puedes presentar un prototipo, que es “es la creación de un modelo operacional que incluya solo algunas características del sistema final” (Kendall & Kendall, 2011).

El concepto de operacional es clave, ya que no se trata de un simple modelo, sino que permite desarrollar módulos que se van integrando en la versión final del sistema.

Esta tercera actividad te permitirá comenzar con la aplicación de los conceptos de programación orientada a objetos (POO). En esta instancia, es fundamental que tu propuesta considere las características principales del lenguaje de programación Java, incluyendo su sintaxis, tipos de datos, estructuras de control y manejo de excepciones.

Dentro de los pilares fundamentales del paradigma que se está empleando, debes realizar un proyecto que incluya correctamente el encapsulamiento, herencia, polimorfismo y abstracción. Estos pilares proporcionan una base sólida para el desarrollo de software modular, flexible y fácilmente mantenible.

Contempla la utilización de estructuras condicionales y de repetición, para controlar el flujo de ejecución de un programa. Adicionalmente, el desarrollo debe incluir un menú con opciones de selección, la declaración y creación de objetos en Java, así como el uso de constructores para inicializar objetos.

Según las características del proyecto, se pueden utilizar estructuras de datos tales como de pilas, colas y listas enlazadas. En este contexto, se puede contemplar, también, la utilización de algoritmos de ordenación y búsqueda. Estas herramientas y conceptos permitirán un desarrollo eficiente, flexible y estructurado del sistema.

Los entregables son los siguientes:

- Explicación del desarrollo en Java.
- Presentación del desarrollo en Java.
- El programa compila y se ejecuta correctamente.

El proyecto o prototipo desarrollado debe contemplar necesariamente las siguientes características:

- Correcta utilización de sintaxis, tipos de datos, estructuras de control.
- Tratamiento y manejo de excepciones.
- Adecuada aplicación del encapsulamiento, herencia, polimorfismo y abstracción.
- Disponibilidad de un menú de selección.
- Empleo de estructuras condicionales y repetitivas.
- Declaración y creación de objetos en Java.
- Utilización de constructores para inicializar objetos.
- Uso de algoritmos de ordenación y búsqueda (opcional según el desarrollo).

Formato Entregable

La actividad propuesta debe ser elaborada en forma individual y respetar los criterios de presentación mencionados a continuación. Se debe realizar una entrega incremental, considerando las correcciones realizadas en la entrega previa.

El informe se debe presentar en un documento en formato PDF (papel A4, letra Calibri 11, espaciado simple). Debe incluir las respuestas a las consignas de forma prolija y ordenada, con su correspondiente explicación para justificar las decisiones adoptadas. Incluir una portada y respetar el siguiente criterio para el nombre del archivo:

«APELLIDO-NOMBRE»-AP3.PDF

La realización de todos los diagramas requiere utilizar una herramienta de libre elección, pero asegurando la prolijidad y claridad de presentación. Todos los diagramas deben incluirse en el documento a entregar con su correspondiente explicación.

Para la persistencia de los datos, deberás utilizar una base de datos MySQL, y para el desarrollo del prototipo, una IDE de Java. La explicación debe estar en el documento a entregar e incluir un enlace a la versión completa (código y archivos asociados) en Github: https://github.com/.

Se evaluará la precisión en las respuestas, capacidad de síntesis, redacción, ortografía y calidad de la presentación.

En caso de requerirlo, es importante revisar el cumplimiento de las normas de estilo American Psychological Association (APA).

Github del proyecto: https://github.com/Valentino-Belli/BELLI-VALENTINO-AP3.git

Correcciones TP2

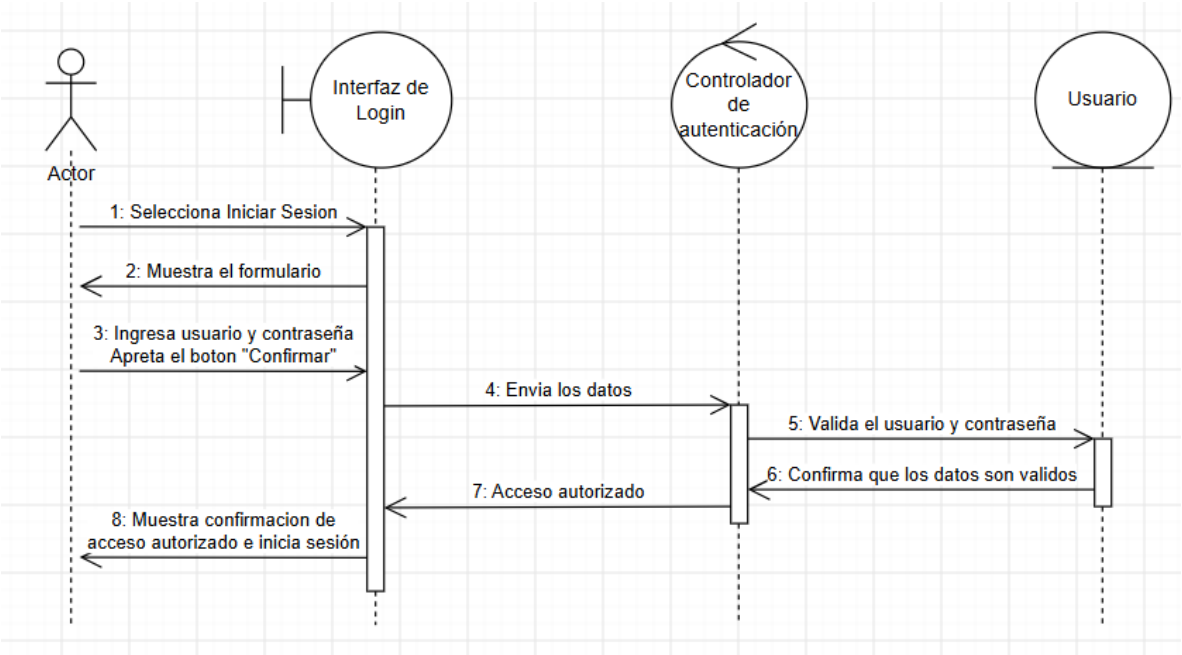
A continuación, se presenta el TP2 con las correcciones realizadas:

1- Etapa de análisis

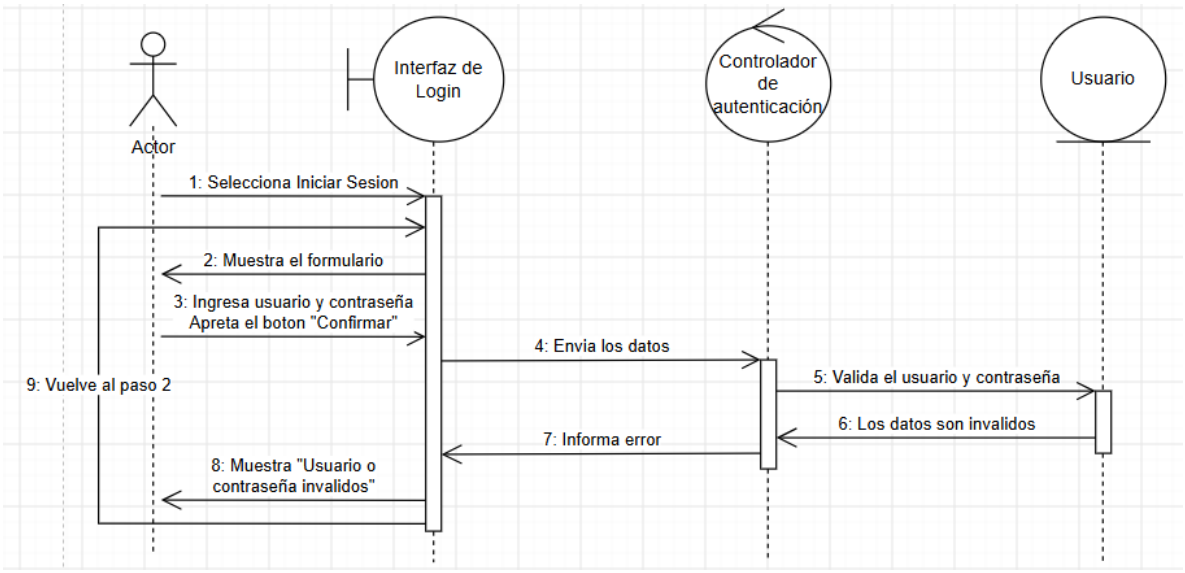
Para empezar con el análisis primero elegimos los casos de uso, luego revisamos los flujos normales y alternativos, y hacemos el diagrama de secuencia. Una vez finalizados los diagramas de secuencia de los casos de uso elegidos, se identifican las diferentes clases que aparecen para luego construir el diagrama de clases del análisis. Por último, se organizan esas clases en paquetes.

Caso de uso: 001 – Iniciar sesión

Flujo normal:

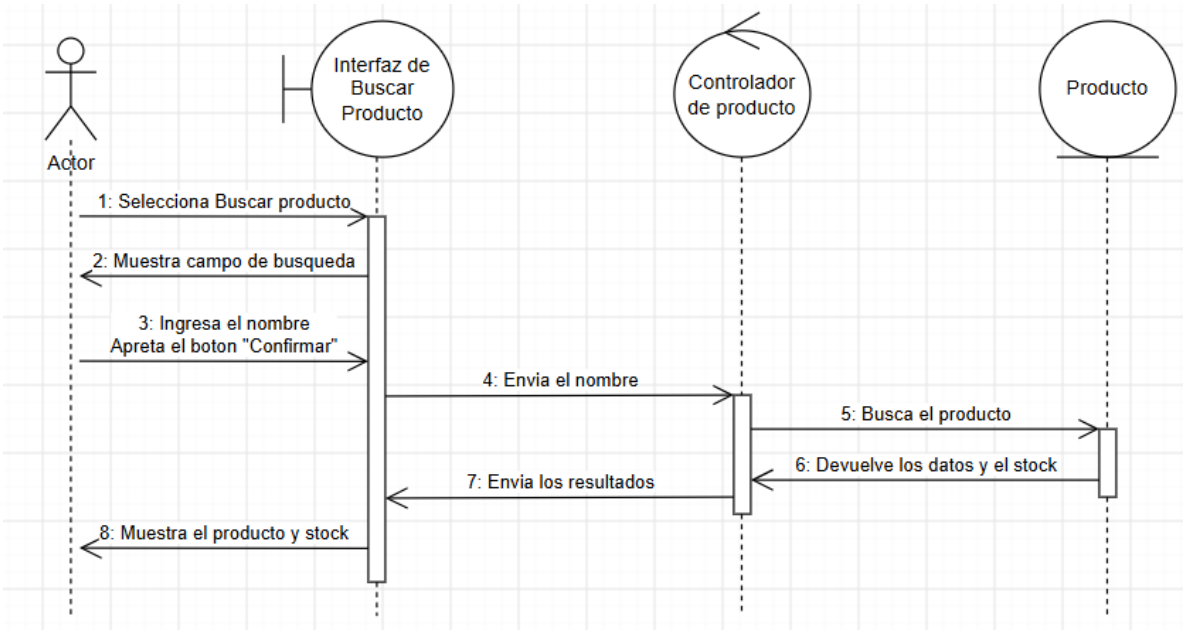


Flujo alternativo: Validación de datos de usuario fallida

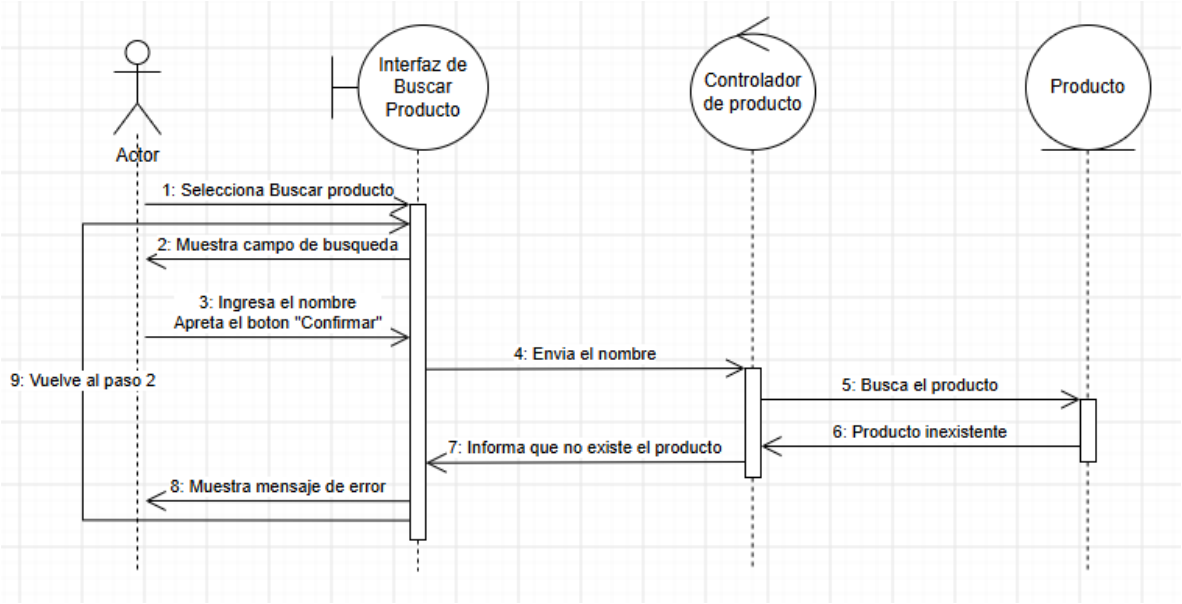


Caso de uso: 002 – Buscar producto

Flujo normal:

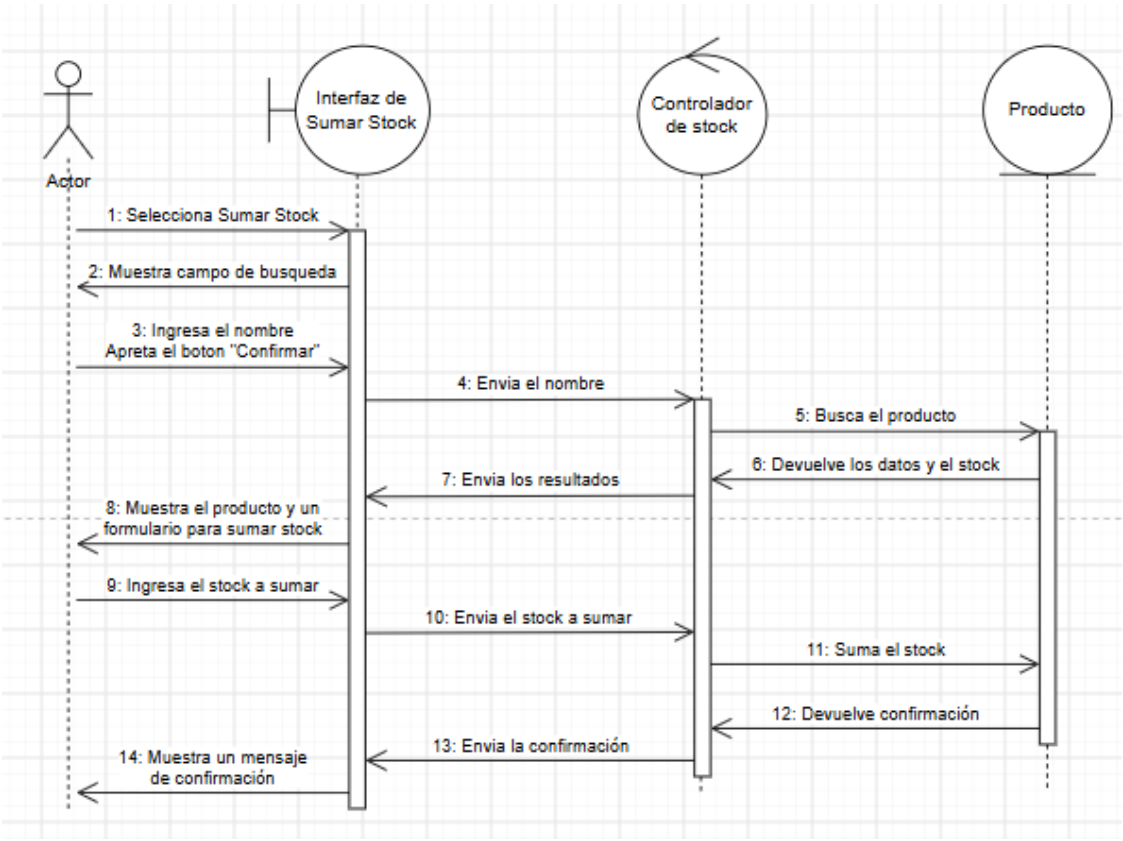


Flujo alternativo: Producto no encontrado

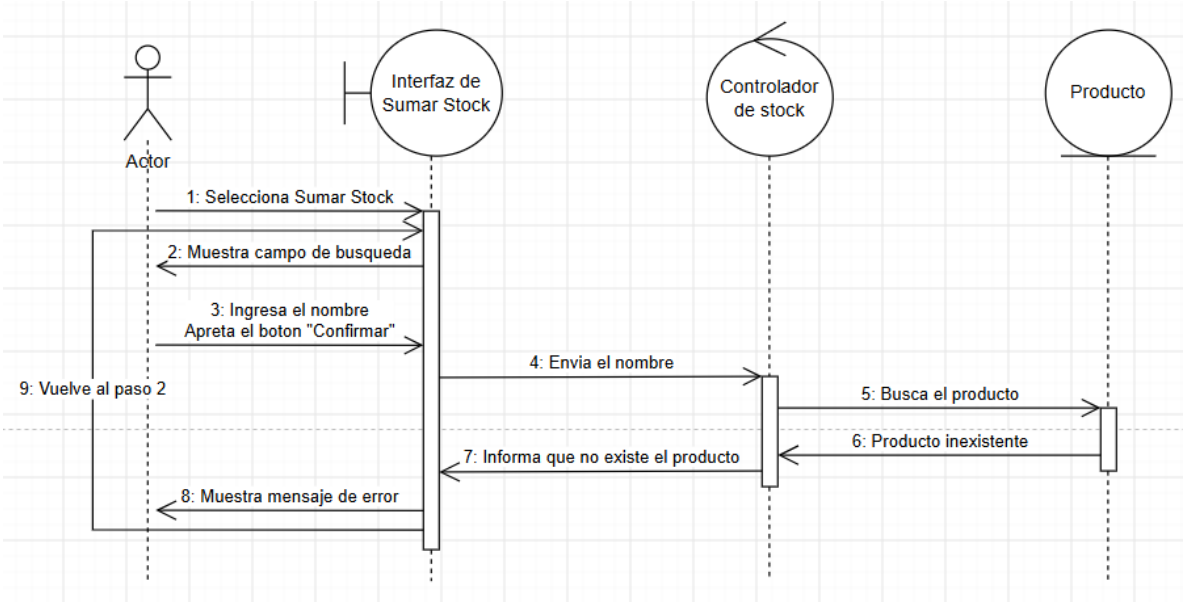


Caso de uso: 003 – Sumar stock

Flujo normal:

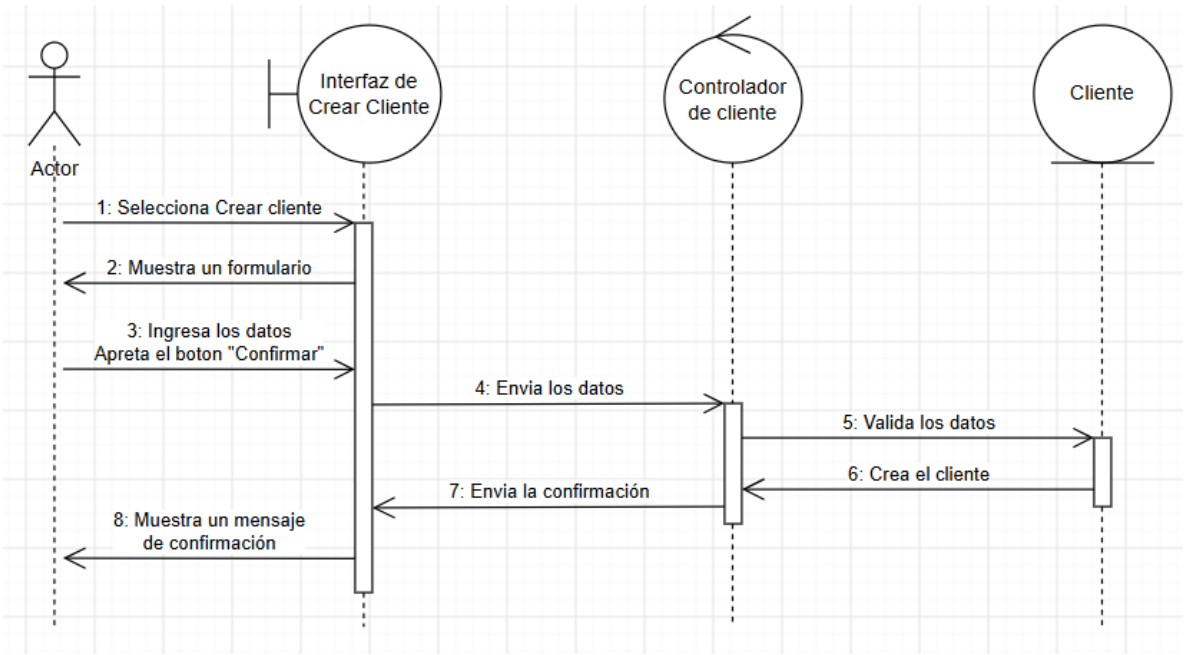


Flujo alternativo: Producto no encontrado

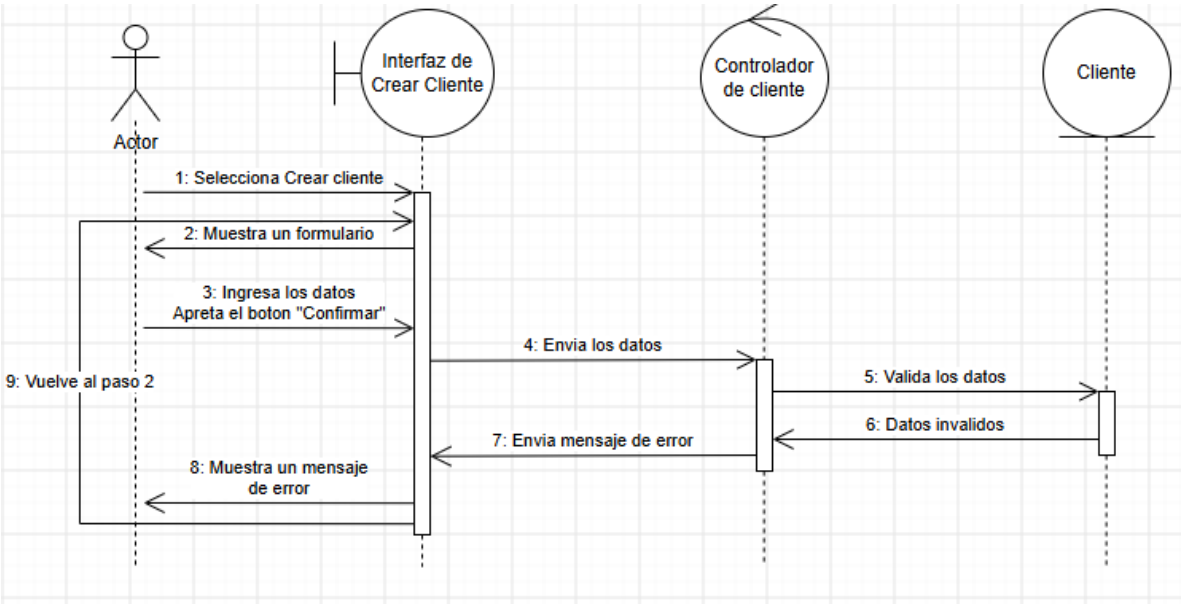


Caso de uso: 004 – Crear cliente

Flujo normal:

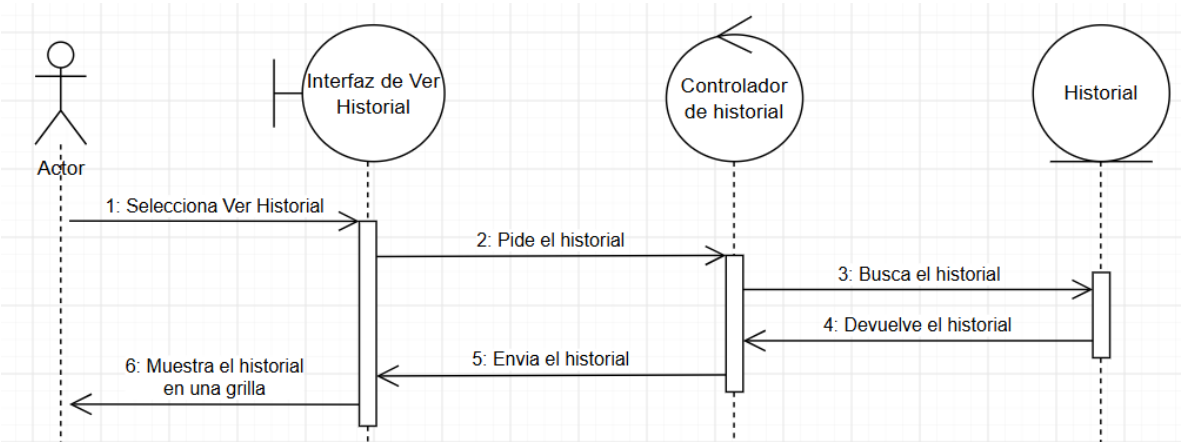


Flujo alternativo: Datos inválidos



Caso de uso: 005 – Ver Historial

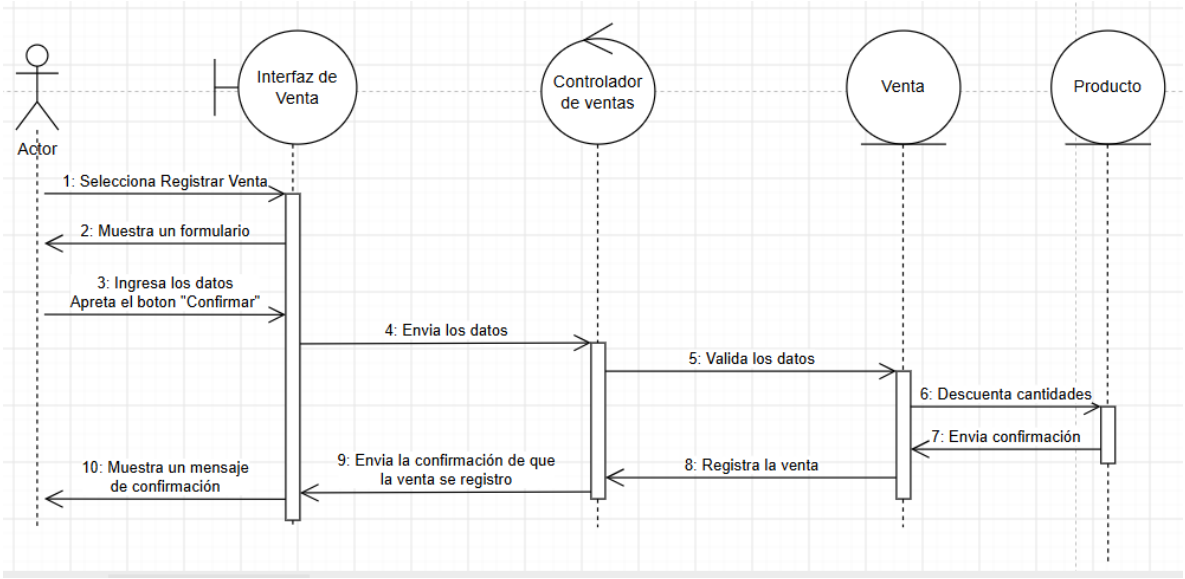
Flujo normal:



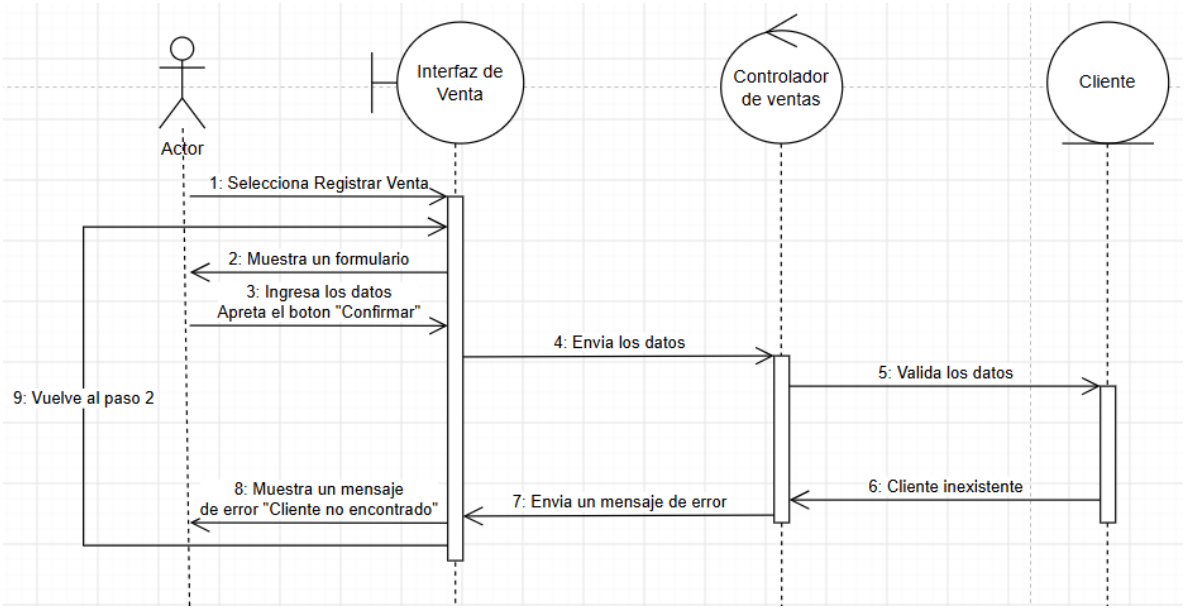
Flujo alternativo: No contempla un flujo alternativo

Caso de uso: 006 – Registrar venta a cliente

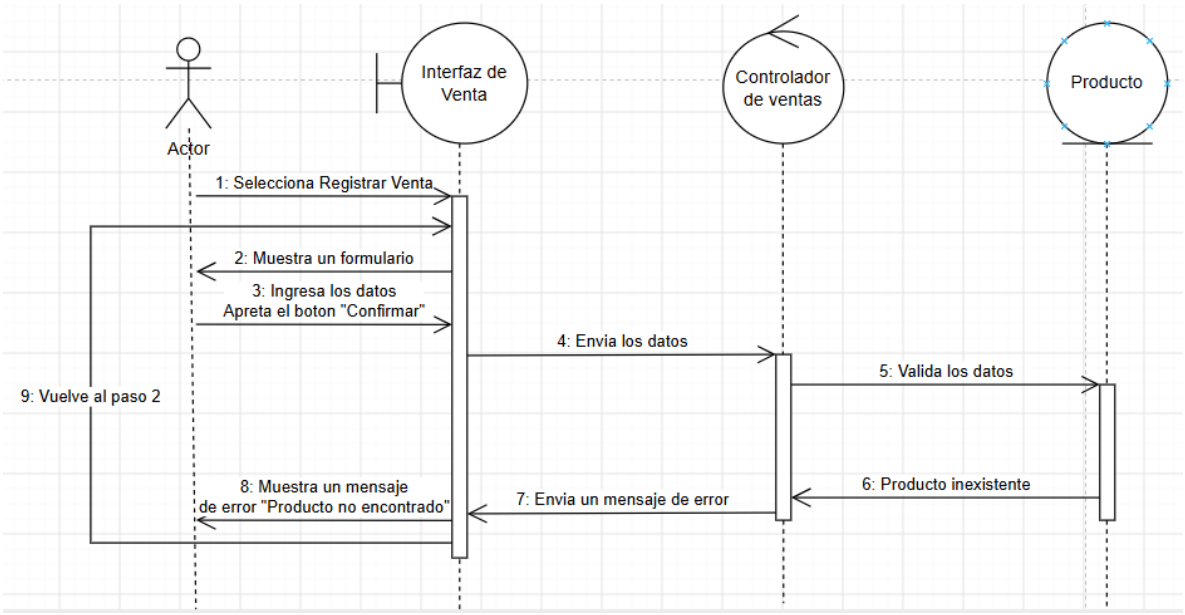
Flujo normal:



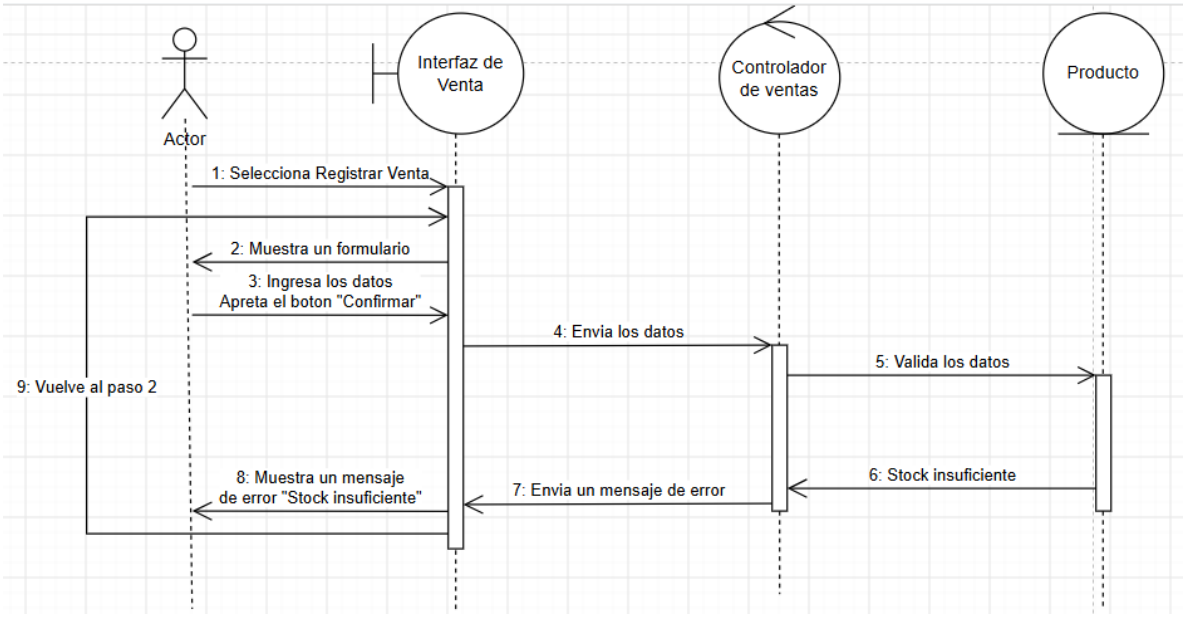
Flujo alternativo: Cliente no encontrado



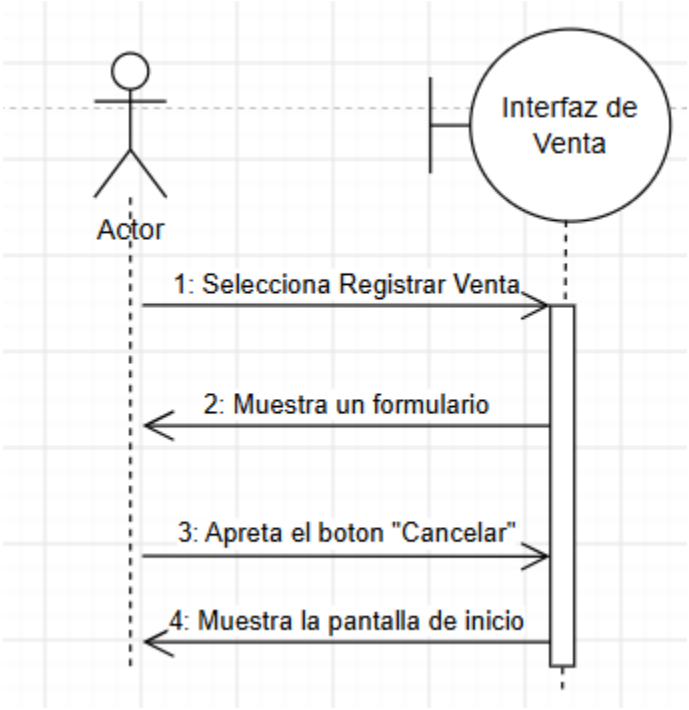
Flujo alternativo: Producto no encontrado



Flujo alternativo: Stock insuficiente

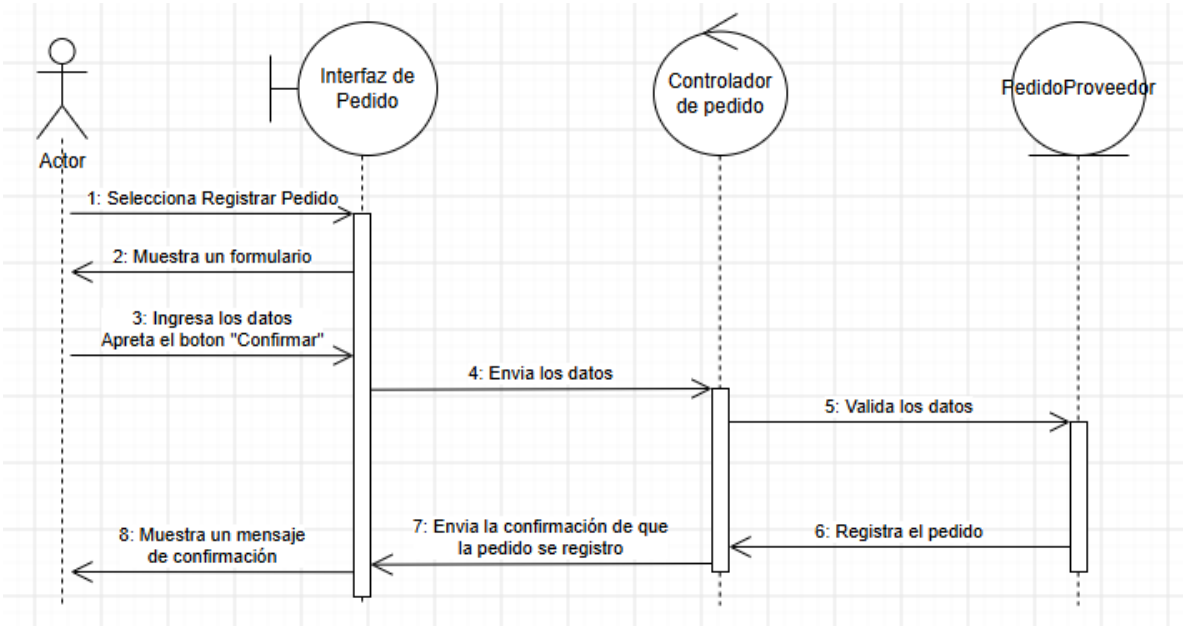


Flujo alternativo: Cancelación de venta

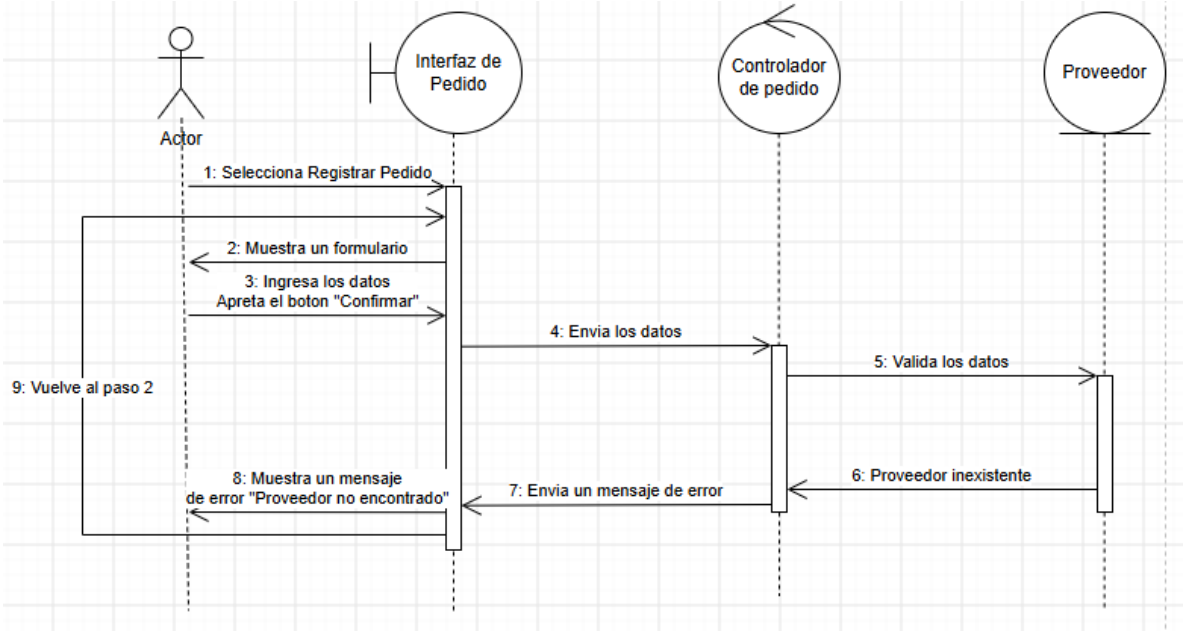


Caso de uso: 007 – Registrar pedido proveedor

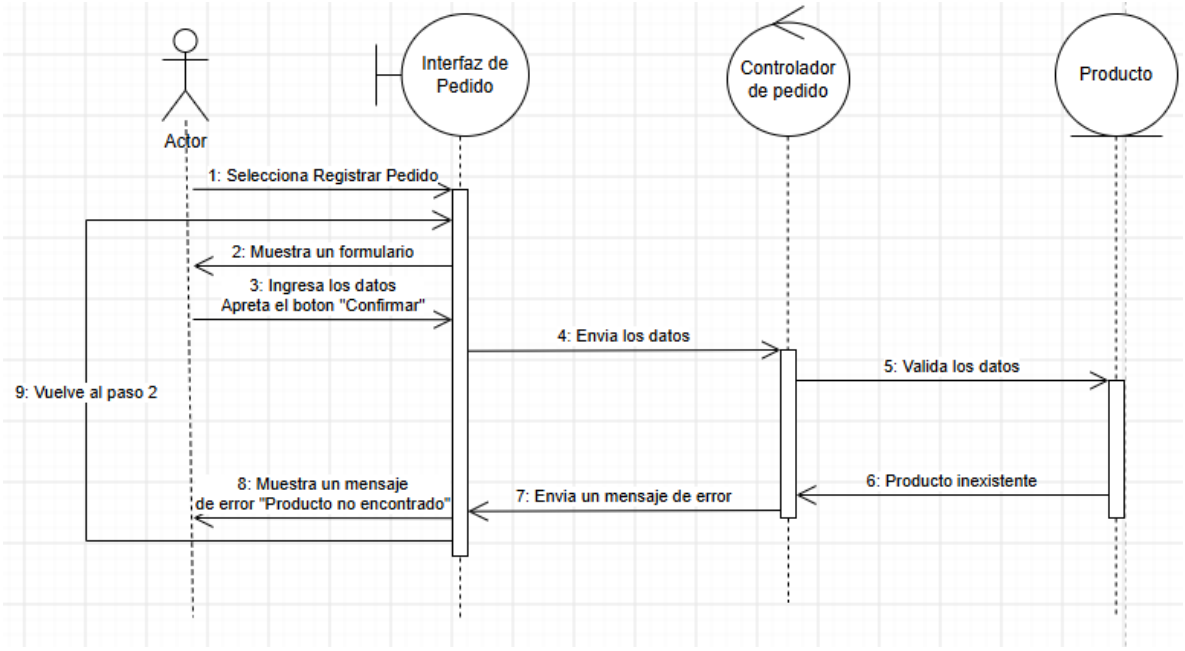
Flujo normal:



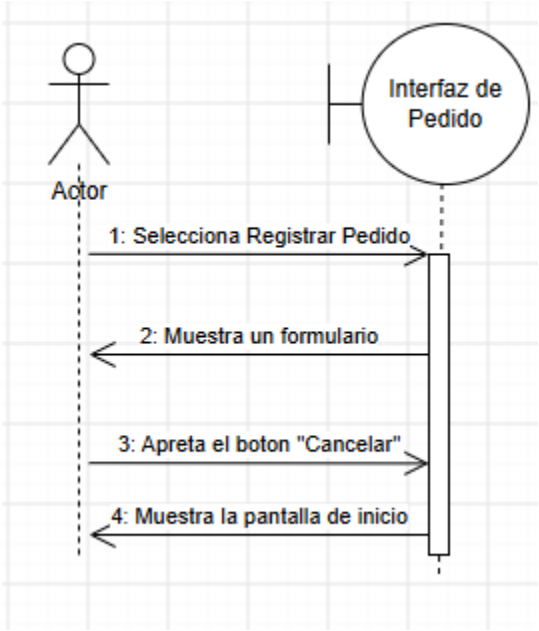
Flujo alternativo: Proveedor no encontrado



Flujo alternativo: Producto no encontrado

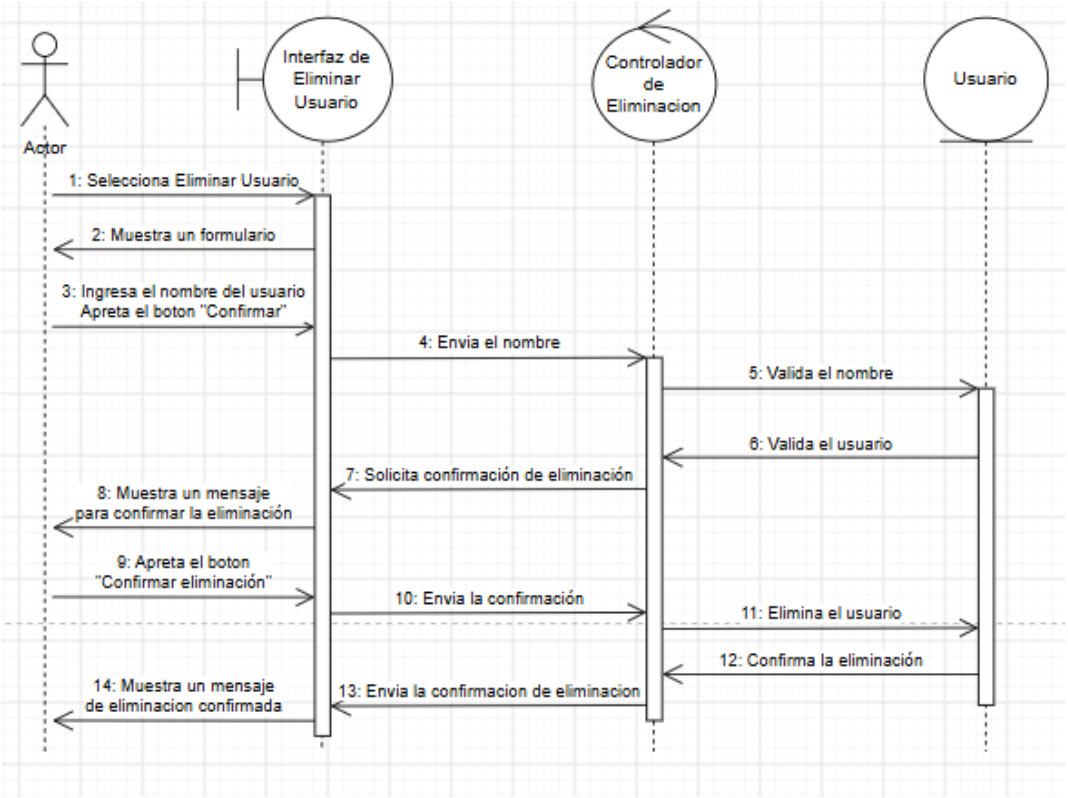


Flujo alternativo: Cancelación de pedido

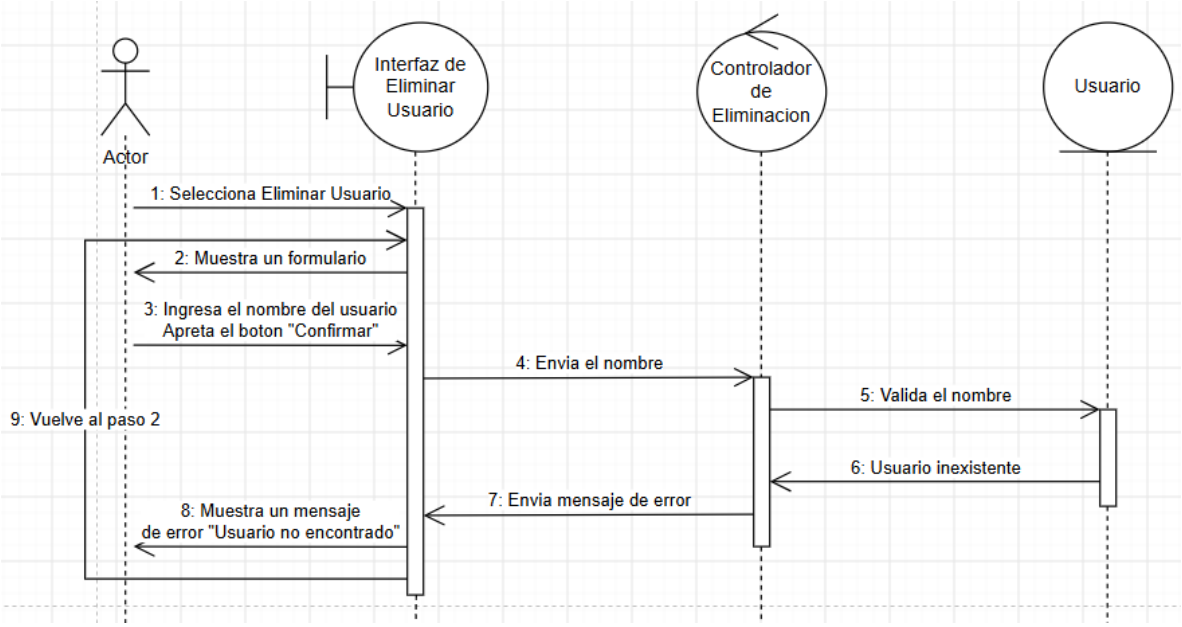


Caso de uso: 008 – Eliminar usuario

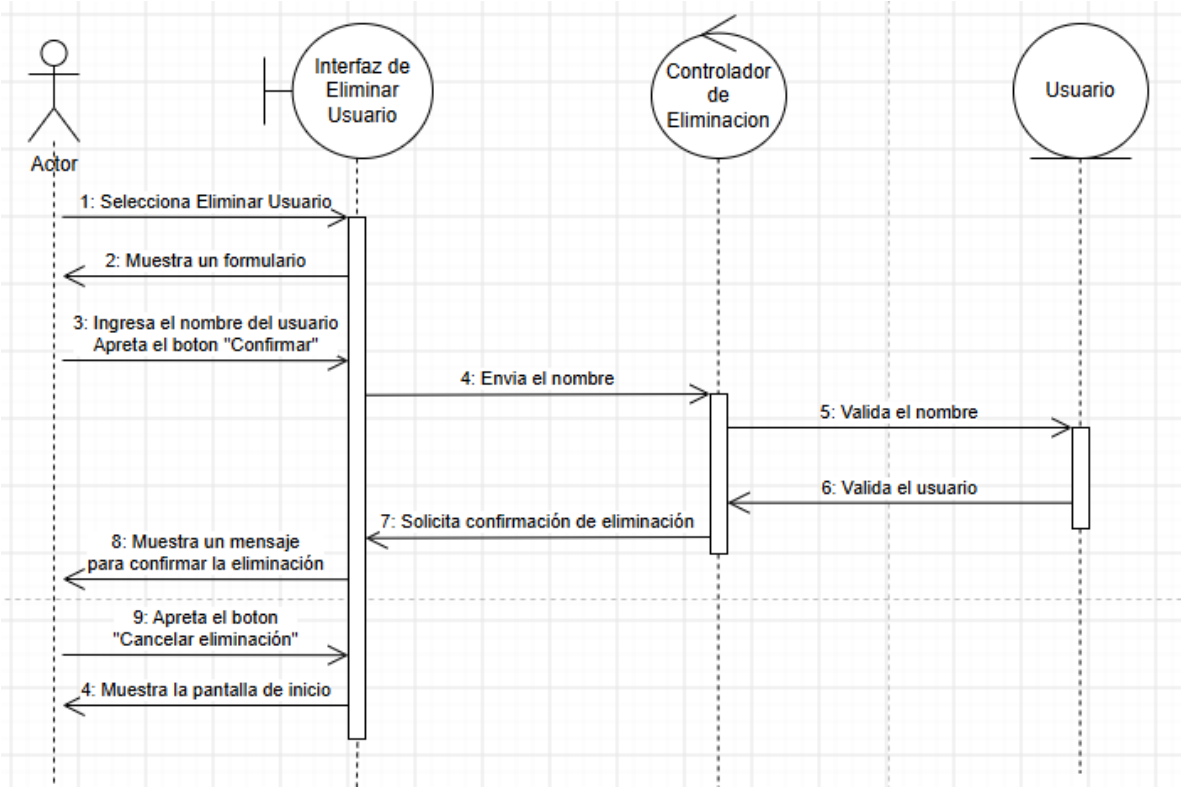
Flujo normal:

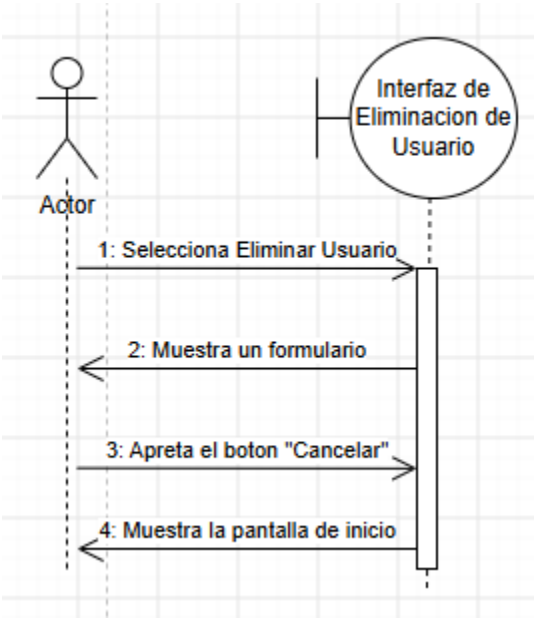


Flujo alternativo: Usuario no encontrado



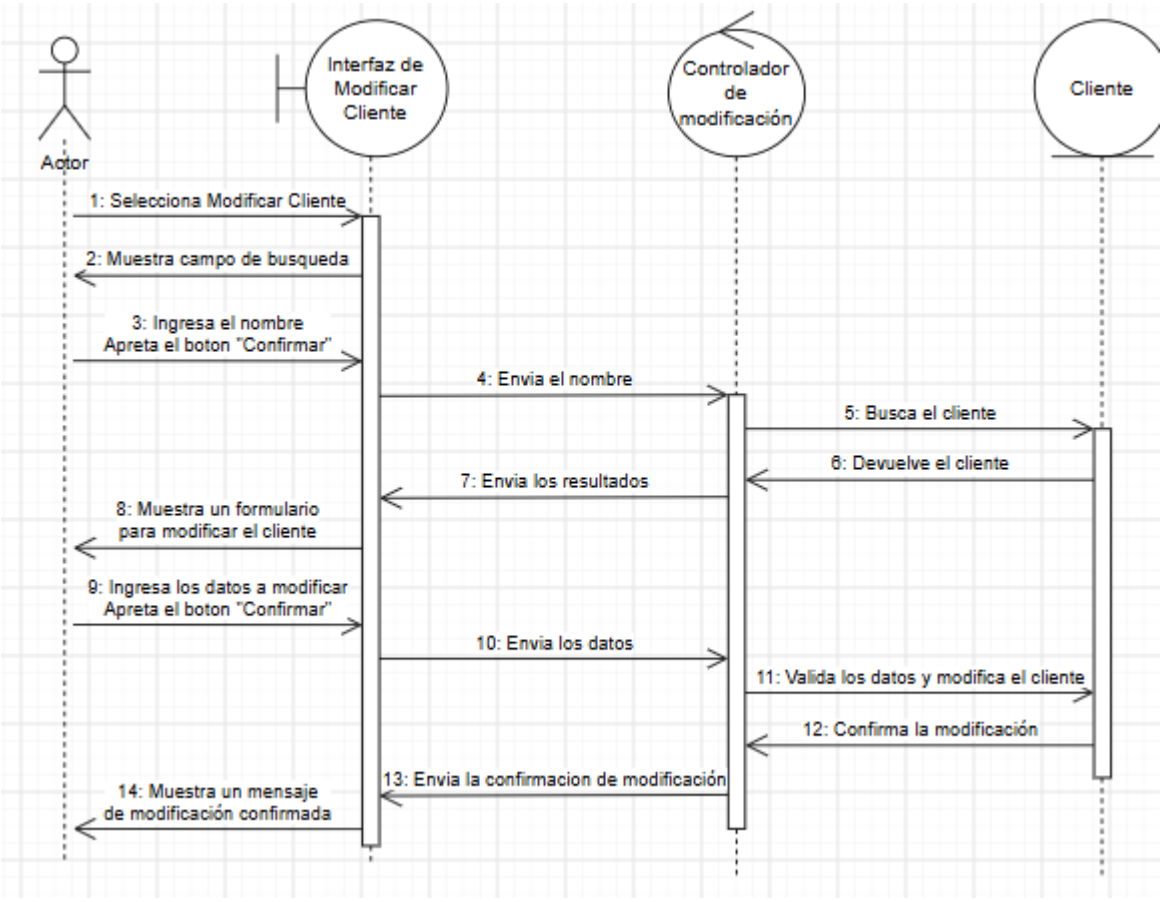
Flujo alternativo: Cancelación de eliminación



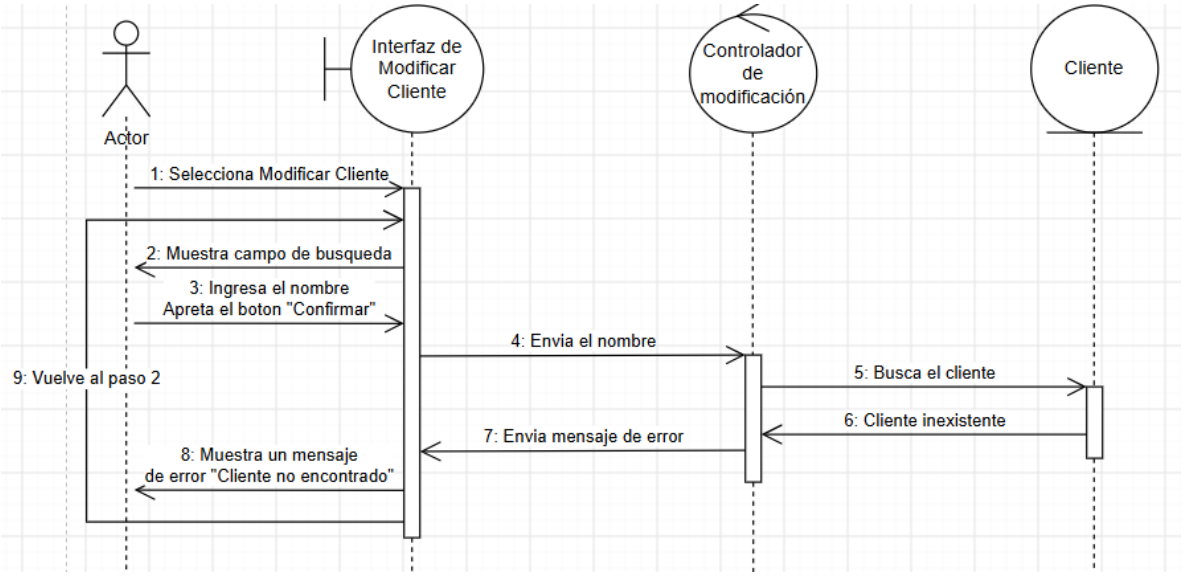


Caso de uso: 009 – Modificar Cliente

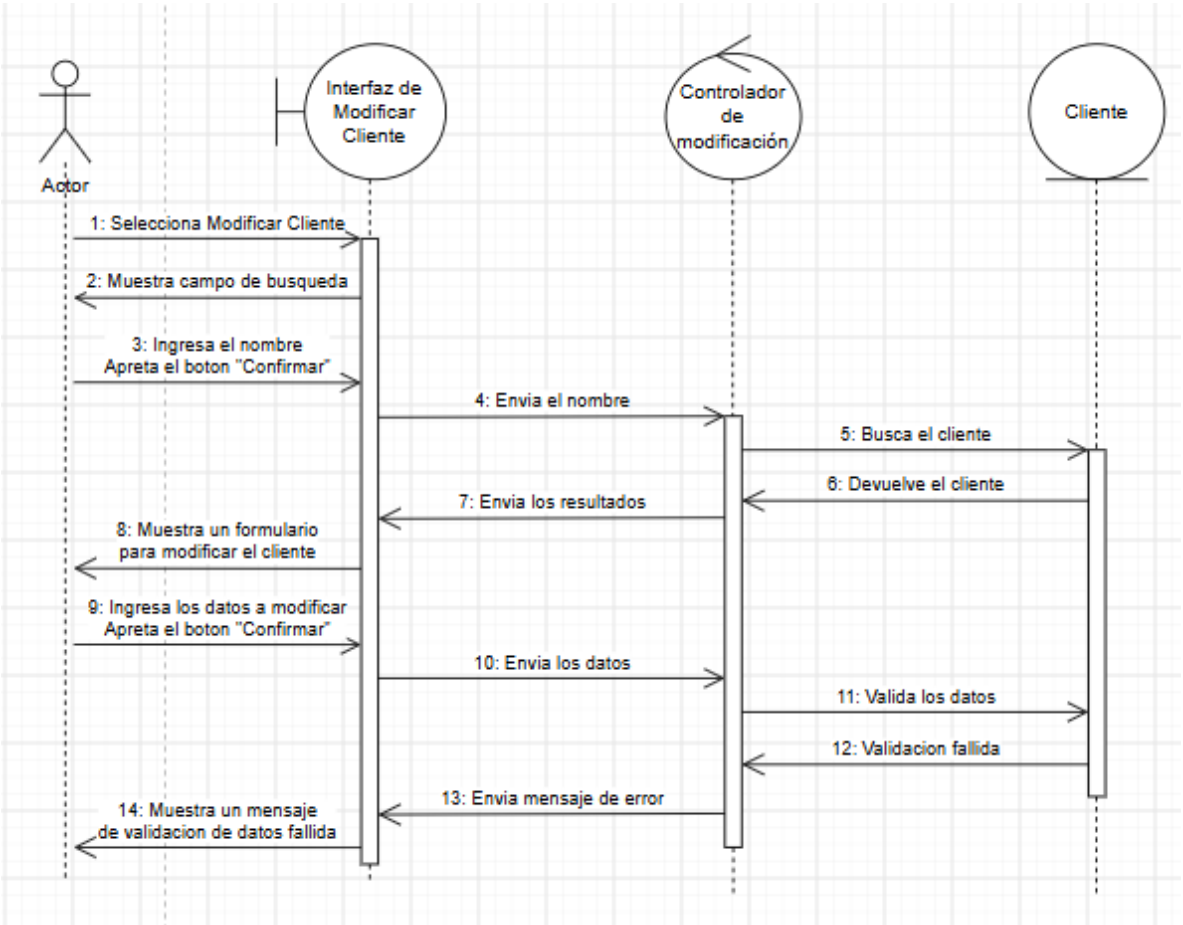
Flujo normal:



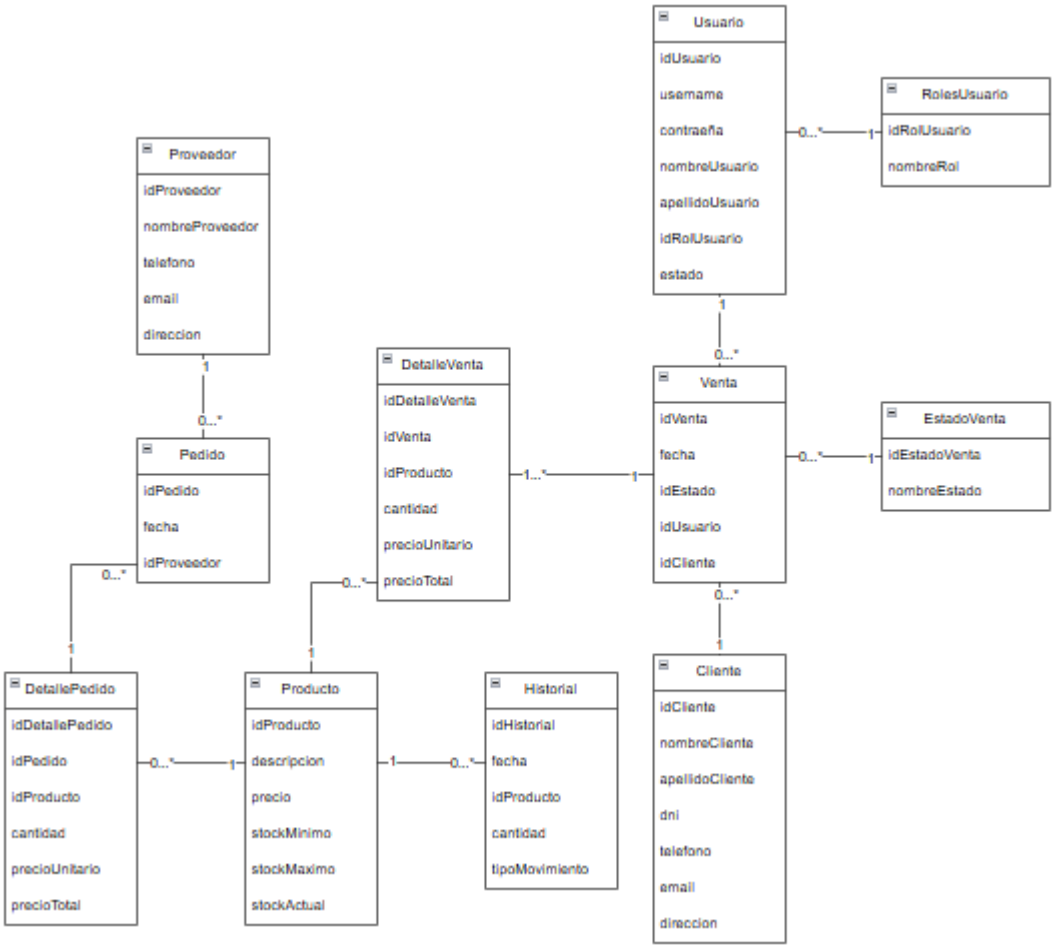
Flujo alternativo: Cliente no encontrado



Flujo alternativo: Validación de datos fallida



El diagrama de clases de análisis es el siguiente:



Los distintos paquetes de análisis son los siguientes:

Paquete	Clases	Descripción
Paquete de seguridad y acceso	Usuario – RolesUsuario	Gestiona la autenticación y los permisos de los usuarios del sistema.
Paquete de ventas	Cliente – Venta – DetalleVenta - EstadoPedido	Administra todo lo relacionado con las ventas de productos y registro de clientes.
Paquete de inventario	Producto – Historial	Controla las existencias de mercadería y los movimientos de almacén.

Paquete de compras	Proveedor – Pedido - DetallePedido	Se encarga del abastecimiento de productos a través de los proveedores oficiales.

2- Etapa de diseño

Para la etapa de diseño debemos hacer el diagrama de clases de diseño, el cual es similar al diagrama de análisis, pero agrega detalles técnicos como tipos de datos y métodos, debido a que está más enfocado en la implementación y desarrollo del sistema.



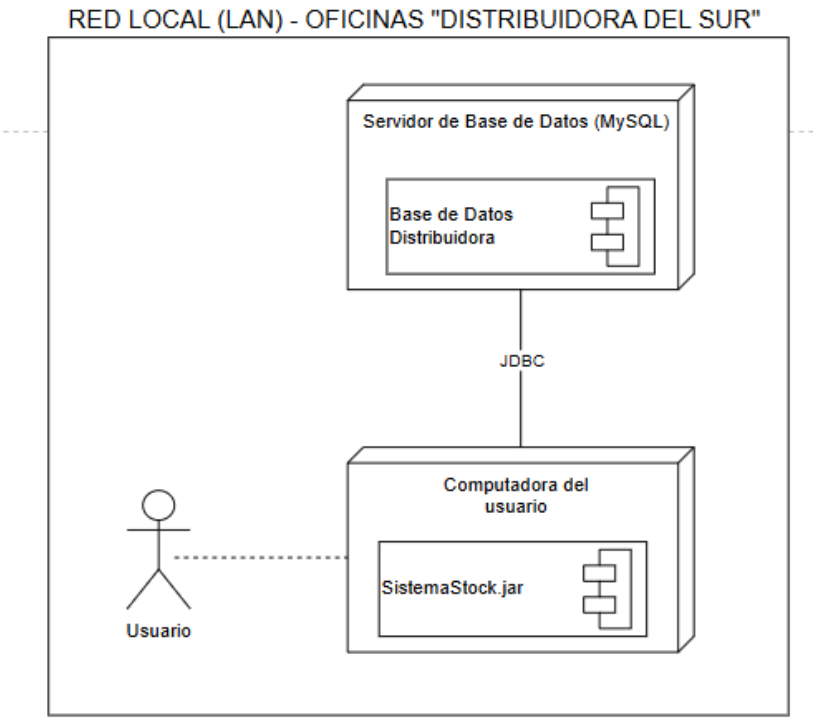
3- Etapa de implementación

Los requerimientos no funcionales (RNF) del sistema son los siguientes.

ID	Descripción
RNF01	El sistema debe estar desarrollado en Java y la interfaz en JavaFX
RNF02	El sistema debe contar con una base de datos MySQL.

RNF03	El sistema debe tener una disponibilidad de 7 x 24, con un tiempo de inactividad de un 1 % en el mes para mantenimiento.
RNF04	Para la persistencia y consulta de datos en la BD se debe utilizar un patrón MVC (modelo-vista-controlador).
RNF05	El sistema debe estar instalado en una infraestructura propia en las oficinas de “Distribuidora del Sur”
RNF06	El sistema debe realizar copias de seguridad (backups) automáticas de la base de datos diariamente fuera del horario comercial
RNF07	Los servidores de la infraestructura propia deben contar con un sistema de energía ininterrumpida (UPS/SAD) para prevenir pérdidas de datos ante cortes eléctricos
RNF08	El sistema debe soportar al menos 15 usuarios operando en simultáneo (administradores, operarios de depósito y facturistas) sin degradar el tiempo de respuesta

En base a los RNF, el diagrama de despliegue del sistema queda de la siguiente manera:



En el nodo “Computadora del usuario” se encuentra el componente SistemaStock.jar, que es el archivo que el usuario instala en la computadora local, en el cual se encuentran las clases de lógica, de interfaz y de acceso a datos.

Luego, este nodo, se conecta mediante el protocolo JDBC al servidor de base de datos, en la cual se encuentra la base de datos de la distribuidora.

4- Etapa de pruebas

4.1.1 Plan de pruebas CU001

Para empezar con la etapa de pruebas, se presenta el plan de pruebas básico propuesto para el caso de uso CU001, “Iniciar Sesión”. Se va a utilizar un código de prueba según la siguiente nomenclatura.

- CP: Caso de prueba.
- Tercera letra: Tipo de prueba.
- Cuarta y quinta letra: Número de prueba.

Caso de uso	Código de prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU001	CPC01	Componente	Cobertura de caminos mínimos	Aplicación de caja blanca al método iniciarSesion(nombre, contraseña) de la clase ControladorUsuario para asegurar que el camino del true (credenciales válidas) y del false (clave errónea o usuario inactivo/inexistente) respondan correctamente.
CU001	CPF02	Componente	Partición de equivalencia / Valores límite.	Aplicación de caja negra al método de autenticación ingresando datos válidos, datos inexistentes, y campos vacíos para comprobar el manejo seguro de excepciones en Java.
CU001	CPS03	Sistema	Pruebas funcionales de interfaz.	Verificación del comportamiento del flujo completo desde la pantalla gráfica. Se espera que al oprimir "Ingresar" con credenciales correctas se abra el menú principal y que ante un error se bloquee el acceso y se muestre un mensaje de alerta al operario.

4.1.2 Modelo de pruebas

Lo primero que se hace es definir las clases de equivalencia y valores frontera para las entradas.

Campo: Nombre de usuario (Longitud permitida: 4 a 20 caracteres)

Equivalencias validas	
CE1	Cadena de texto con longitud entre 4 y 20 caracteres, existente en la BD.

Equivalencias invalidas	
CE2	Campo vacío (longitud 0).
CE3	Cadena fuera de los límites permitidos (menor a 4 o mayor a 20 caracteres).
CE4	Nombre de usuario que no existe en la base de datos de la distribuidora.

Campo: Contraseña (Longitud permitida: 6 a 15 caracteres)

Equivalencias validas	
CE5	Contraseña correcta que coincide con el usuario en la BD.

Equivalencias invalidas	
CE6	Campo vacío (longitud 0).
CE7	Contraseña incorrecta para este usuario
CE8	Cadena fuera de los límites permitidos (menor a 6 o mayor a 15 caracteres).

Clase/Frontera	Datos de entrada (Usuario y contraseña)	Comportamiento esperado	Mensaje emitido por el sistema
CE1 + CE5 (Éxito)	Admin / cLAvE132	Permite el acceso. Cierra el login y abre el inicio del programa	“Inicio de sesión exitoso. Bienvenido al sistema.”

CE2 (Campo “username” vacío)	[Vacío] / cLAvE132	Bloquea el envío. Foco en “username”	“Error: El campo 'Nombre de usuario' no puede estar vacío."
CE6 (Campo “contraseña” vacío)	Admin / [Vacío]	Bloquea el envío. Foco en “contraseña”	“Error: El campo 'Contraseña’ no puede estar vacío.”
CE3 (Frontera inferior)	Adm / cLAvE132	Rechazado en el cliente por el formato	“Error: El usuario debe tener entre 4 y 20 caracteres.”
CE3 (Frontera superior)	AdministradorDelSistema / cLAvE132	Rechazado en el cliente por el formato	“Error: El usuario debe tener entre 4 y 20 caracteres.”
CE4 (Usuario inexistente)	usuarioFalso / cLAvE132	Falla la autenticación en el controlador.	"Error: Usuario o contraseña incorrectos."
CE7 (Contraseña incorrecta)	Admin / ClaVe978	Falla la autenticación en el controlador.	"Error: Usuario o contraseña incorrectos."
CE8 (Frontera inferior)	Admin / cLAvE	Rechazado en el cliente por el formato	“Error: La contraseña debe tener entre 6 y 15 caracteres.”
CE8 (Frontera superior)	Admin / cLAvEsEgUrA132465	Rechazado en el cliente por el formato	“Error: La contraseña debe tener entre 6 y 15 caracteres.”

Algunos de los casos de prueba para el caso de uso C001 “Iniciar Sesión” son los siguientes:

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-001	Funcional	Verificar que un operario activo con credenciales correctas pueda ingresar al sistema.	El usuario operario1 debe estar registrado en la base de datos y el valor usuarioActivo = true.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	Usuario: operario1 Contraseña: Stock123	Acceder al sistema. Ingresar los datos en sus respectivos campos. Presionar “Ingresar”	El método retorna true, la pantalla de login se cierra y se despliega el menú principal.

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-002	Funcional	Verificar que el sistema deniegue el acceso ante una contraseña inválida.	El usuario operario1 debe estar registrado en la base de datos y el valor usuarioActivo = true.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	Usuario: operario1 Contraseña: ClaveErronea	Acceder al sistema. Ingresar los datos en sus respectivos campos. Presionar “Ingresar”	El método retorna false, permanece en la pantalla de login y se muestra una ventana emergente con el mensaje de la clase de equivalencia CE7.

4.1.3 Tratamiento de defectos

Este artefacto registra las fallas encontradas durante la ejecución de las pruebas. Algunos ejemplos de posibles defectos que se podrían encontrar en un caso de uso “Iniciar Sesión” son estas:

Defecto ID	DEF-CU001-01
Caso de Prueba Asociado	CP-002 (Intento de inicio de sesión con Contraseña Incorrecta).
Descripción del Defecto	Al ingresar una contraseña incorrecta, el sistema lanza una excepción en la consola de Java (NullPointerException en ControladorUsuario.java:42) en lugar de mostrar la alerta gráfica. La pantalla se queda congelada y el operario no sabe qué pasó.
Severidad	Alta (Bloquea la operación normal de manejo de errores del sistema).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se añadió un bloque try-catch en el método iniciarSesion del controlador para capturar la respuesta nula de la base de datos cuando las credenciales no coinciden. Ahora el método retorna false limpiamente y dispara la alerta gráfica en la interfaz ("Error: Usuario o contraseña incorrectos.").

Defecto ID	DEF-CU001-02
Caso de Prueba Asociado	CE4 (Nombre de usuario demasiado largo).
Descripción del Defecto	Si el operario escribe un nombre de usuario de más de 20 caracteres, la interfaz gráfica permite enviarlo, lo que provoca un colapso en la base de datos MySQL (Data truncation: Data too long for column 'nombre_usuario').
Severidad	Media (Provoca una caída controlable, pero expone datos internos del motor SQL).
Prioridad de Resolución	Media
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación de longitud del lado del cliente (en la pantalla) antes de invocar al controlador. Si supera los 20

	caracteres, el botón "Ingresar" no se activa y se muestra un texto de advertencia en rojo.

4.1.4 Evaluación de pruebas

Las pruebas permitieron comprobar que el sistema permite iniciar sesión correctamente cuando se ingresan un nombre de usuario y una contraseña válidos. También se verificó que, ante el ingreso de credenciales incorrectas, el sistema informa adecuadamente el error y permite al usuario volver a ingresar sus datos.

Durante la ejecución de las pruebas se identificaron distintos defectos, los cuales fueron registrados y tratados. Una vez realizadas las correcciones, se repitieron las pruebas afectadas para verificar que los errores hayan sido solucionados y que las modificaciones no afectaran el funcionamiento esperado.

Como resultado final, las pruebas realizadas para el caso de uso CU001 – Iniciar Sesión fueron satisfactorias, cumpliéndose los resultados esperados definidos para los casos de prueba.

4.2.1 Plan de pruebas CU002

El siguiente plan de pruebas básico propuesto es para el caso de uso CU002, “Buscar Producto”.

Caso de uso	Código de prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU002	CPC01	Componente	Cobertura de caminos mínimos	Aplicación de caja blanca al método buscarProducto(id) de la clase ControladorProducto para asegurar que, si el producto existe, el método retorne el objeto de tipo “Producto” correspondiente. En caso de no existir el producto, retornar el valor “null”.
CU002	CPF02	Componente	Partición de equivalencia / Valores límite.	Aplicación de caja negra al método de busqueda de producto ingresando el dato valido, datos inexistentes, y campos vacíos para comprobar el manejo seguro de excepciones en Java.
CU002	CPS03	Sistema	Pruebas funcionales de interfaz.	Verificación del comportamiento del flujo completo desde la pantalla gráfica. Se espera que al oprimir "Buscar" con un id existente se muestren los detalles del producto y que ante un error solamente se muestre un mensaje de alerta al operario.

4.2.2 Modelo de pruebas

Las clases de equivalencia y valores frontera son las siguientes:

Campo: id (Tipo int)

Equivalencias validas	
CE1	Número entero mayor a 0, existente en la BD.

Equivalencias invalidas	
CE2	Campo vacío (longitud 0).
CE3	Número fuera de los límites permitidos (id≤0 o valores no enteros)
CE4	Número que no existe en la base de datos de la distribuidora.

Clase/Frontera	Datos de entrada (id)	Comportamiento esperado	Mensaje emitido por el sistema
CE1 (Éxito)	33	Busca y encuentra el objeto. Cierra el buscador y abre un formulario no editable con los datos del producto.	“Producto encontrado”
CE2 (Campo “id” vacío)	[Vacío]	Bloquea el envío. Foco en “id”	“Error: El campo 'id' no puede estar vacío."
CE1 (Frontera de los valores permitidos)	1	Busca y encuentra el objeto. Cierra el buscador y abre un formulario no editable con los datos del producto.	“Producto encontrado”
CE3 (Frontera de los valores no permitidos)	0	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”

CE3	-5	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE3	3.3	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE3	-3.3	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE4 (Producto inexistente)	999	Falla la busqueda en el controlador.	"Error: El producto no existe."

Algunos de los casos de prueba para el caso de uso C002 “Buscar Producto” son los siguientes:

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-001	Funcional	Verificar que un operario pueda buscar un producto existente.	El producto con id=1 debe estar registrado en la base de datos.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	id: 1	Acceder al sistema. Ingresar a la interfaz de buscar producto. Ingresar el id en su respectivo campo. Presionar “Buscar”	El método retorna el producto, la pantalla muestra un formulario no editable con los datos del producto.

Nro. de caso de prueba	Tipo de prueba	Descripción	Precondiciones
CP-002	Funcional	Verificar que el sistema muestre un mensaje de error al ingresar un id invalido.	El producto con id=-5 no se encuentra registrado en la base de datos.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	id: -5	Acceder al sistema. Ingresar a la interfaz de buscar producto. Ingresar el id en su respectivo campo. Presionar “Buscar”	El método retorna null, como consecuencia, se muestra el mensaje de la clase de equivalencia CE3.

4.2.3 Tratamiento de defectos

Algunos ejemplos de posibles defectos que se podrían encontrar en un caso de uso “Buscar Producto” son estos:

Defecto ID	DEF-CU002-01
Caso de Prueba Asociado	CE4 (Intento de buscar un producto con un id inexistente en la base de datos).
Descripción del Defecto	Al ingresar un id inexistente, el sistema lanza una excepción en la consola de Java (NullPointerException en ControladorProducto.java:42) en lugar de mostrar la alerta gráfica. La pantalla se queda congelada y el operario no sabe qué pasó.
Severidad	Alta (Bloquea la operación normal de manejo de errores del sistema).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se añadió un bloque try-catch en el método BuscarProducto del controlador para capturar la respuesta nula de la base de datos cuando el producto no es encontrado. Ahora el método retorna false limpiamente y dispara la alerta gráfica en la interfaz ("Error: Producto no encontrado.").

Defecto ID	DEF-CU002-02
Caso de Prueba Asociado	CE3 (id con valor de punto flotante).
Descripción del Defecto	Si el operario escribe un id con decimales, la interfaz gráfica permite enviarlo, lo que provoca un colapso en la base de datos MySQL. (ERROR 1265 (01000): Data truncated for column 'id' at row 1).
Severidad	Media (Provoca una caída controlable, pero expone datos internos del motor SQL).
Prioridad de Resolución	Media
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación de tipo de dato del lado del cliente (en la pantalla) antes de invocar al controlador. Si el valor tiene decimales, es 0 o negativo, el botón "Buscar" no se activa y se muestra un texto de advertencia en rojo.

4.2.4 Evaluación de pruebas

Las pruebas permitieron comprobar que el sistema permite buscar un producto correctamente cuando se ingresan un id válido. También se verificó que, ante el ingreso de un id incorrecto, el sistema informa adecuadamente el error y permite al usuario volver a ingresar el id.

Durante la ejecución de las pruebas se identificaron distintos defectos, los cuales fueron registrados y tratados. Una vez realizadas las correcciones, se repitieron las pruebas afectadas para verificar que los errores hayan sido solucionados y que las modificaciones no afectaran el funcionamiento esperado.

Como resultado final, las pruebas realizadas para el caso de uso CU002 – Buscar Producto fueron satisfactorias, cumpliéndose los resultados esperados definidos para los casos de prueba.

4.3.1 Plan de pruebas CU003

El siguiente plan de pruebas básico propuesto es para el caso de uso CU003, “Sumar Stock”.

Caso de uso	Código de prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU003	CPC01	Componente	Cobertura de caminos mínimos	Aplicación de caja blanca al método registrarIngresoStock(id, cantidad) de la clase ControladorProducto para asegurar que el método sume correctamente el stock del producto seleccionado.
CU003	CPF02	Componente	Partición de equivalencia / Valores límite.	Aplicación de caja negra al método de autenticación ingresando datos válidos, datos inexistentes, y campos vacíos para comprobar el manejo seguro de excepciones en Java.
CU003	CPS03	Sistema	Pruebas funcionales de interfaz.	Verificación del comportamiento del flujo completo desde la pantalla gráfica. Se espera que al oprimir "Sumar stock" con datos correctos se sume el stock y el sistema vuelve a la pantalla de inicio, y que ante un error se muestre un mensaje de alerta al operario.

4.3.2 Modelo de pruebas

Las clases de equivalencia y valores frontera son las siguientes:

Campo: id (Tipo int)

Equivalencias validas	
CE1	Número entero mayor a 0, existente en la BD.

Equivalencias invalidas	
CE2	Campo vacío (longitud 0).
CE3	Número fuera de los límites permitidos (id≤0 o valores no enteros)
CE4	Número que no existe en la base de datos de la distribuidora.

Campo: cantidad (Tipo int)

Equivalencias validas	
CE5	Número entero mayor a 0.

Equivalencias invalidas	
CE6	Campo vacío (longitud 0).
CE7	Número fuera de los límites permitidos (id≤0 o valores no enteros)

Clase/Frontera	Datos de entrada (id y cantidad)	Comportamiento esperado	Mensaje emitido por el sistema
CE1 y CE5 (Éxito)	33 / 5	Busca y encuentra el objeto. Suma el stock. Vuelve a la interfaz de inicio.	“Producto encontrado y stock sumado correctamente”
CE2 (Campo “id” vacío)	[Vacío] / 5	Bloquea el envío. Foco en “id”	“Error: El campo 'id' no puede estar vacío."
CE6 (Campo “cantidad” vacío)	33 / [Vacío]	Bloquea el envío. Foco en “cantidad”	“Error: El campo 'cantidad' no puede estar vacío."

CE3 (Frontera de los valores permitidos)	1 / 5	Busca y encuentra el objeto. Suma el stock. Vuelve a la interfaz de inicio.	“Producto encontrado y stock sumado correctamente”
CE3 (Frontera de los valores no permitidos)	0 / 5	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE3	-5 / 5	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE3	3.3 / 5	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE3	-3.3 / 5	Rechazado en el cliente por el formato	“Error: El id debe ser un número entero mayor a 0.”
CE7 (Frontera de los valores permitidos)	33 / 1	Busca y encuentra el objeto. Suma el stock. Vuelve a la interfaz de inicio.	“Producto encontrado y stock sumado correctamente”
CE7 (Frontera de los valores no permitidos)	33 / 0	Rechazado en el cliente por el formato	“Error: La cantidad debe ser un número entero mayor a 0.”
CE7	33 / -5	Rechazado en el cliente por el formato	“Error: La cantidad debe ser un número entero mayor a 0.”
CE7	33 / 3.3	Rechazado en el cliente por el formato	“Error: La cantidad debe ser un

			número entero mayor a 0.”
CE7	33 / -3.3	Rechazado en el cliente por el formato	“Error: La cantidad debe ser un número entero mayor a 0.”
CE4 (Producto inexistente)	999	Falla la busqueda en el controlador.	"Error: El producto no existe."

Algunos de los casos de prueba para el caso de uso C003 “Sumar Stock” son los siguientes:

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-001	Funcional	Verificar que un operario pueda buscar un producto existente y sumarle stock.	El producto con id=1 debe estar registrado en la base de datos.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	id: 1 cantidad: 5	Acceder al sistema. Ingresar a la interfaz de sumar stock. Ingresar el id y la cantidad en sus respectivos campos. Presionar “Sumar stock”	El método busca el producto y le suma al stock la cantidad ingresada. La pantalla muestra el mensaje “Stock sumado correctamente” y vuelve a la interfaz de inicio.

Nro. de caso de prueba	Tipo de prueba	Descripción	Precondiciones
CP-002	Funcional	Verificar que el sistema muestre un mensaje de error al ingresar un id invalido.	El producto con id=-5 no está registrado en la base de datos.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	id: -5 cantidad: 5	Acceder al sistema. Ingresar a la interfaz de sumar stock. Ingresar el id y la cantidad en sus respectivos campos. Presionar “Sumar stock”	El método muestra el mensaje de la clase de equivalencia CE3.

4.3.3 Tratamiento de defectos

Algunos ejemplos de posibles defectos que se podrían encontrar en un caso de uso “Sumar Stock” son estas:

Defecto ID	DEF-CU003-01
Caso de Prueba Asociado	CE4 (Intento de sumarle stock a un producto inexistente).
Descripción del Defecto	Al ingresar un id inexistente, el sistema lanza una excepción en la consola de Java (NullPointerException en ControladorProducto.java:42) en lugar de mostrar la alerta gráfica. La pantalla se queda congelada y el operario no sabe qué pasó.
Severidad	Alta (Bloquea la operación normal de manejo de errores del sistema).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se añadió un bloque try-catch en el método registrarIngresoStock del controlador para capturar la respuesta nula de la base de datos cuando el producto no es encontrado. Ahora el método retorna false limpiamente y dispara la alerta gráfica en la interfaz ("Error: Producto no encontrado.").

Defecto ID	DEF-CU003-02
Caso de Prueba Asociado	CE7 (cantidad negativa).
Descripción del Defecto	Si el operario ingresa una cantidad negativa, la interfaz gráfica permite enviarlo, pero, al ser un valor negativo, en vez de sumar el stock, éste se resta, generando un error de lógica del negocio.
Severidad	Alta (No provoca ningún error técnico o de falla de sistema, sino que resta stock en vez de sumarlo, generando datos erróneos en el sistema).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación del lado del cliente (en la pantalla) antes de invocar al controlador. Si el valor tiene decimales, es 0 o negativo, el botón "Sumar Stock" no se activa y se muestra un texto de advertencia en rojo.

4.3.4 Evaluación de pruebas

Las pruebas permitieron comprobar que el sistema permite sumar stock correctamente cuando se ingresa un id y una cantidad válida. También se verificó que, ante el ingreso de datos incorrectos, el sistema informa adecuadamente el error y permite al usuario volver a ingresar los datos.

Durante la ejecución de las pruebas se identificaron distintos defectos, los cuales fueron registrados y tratados. Una vez realizadas las correcciones, se repitieron las pruebas afectadas para verificar que los errores hayan sido solucionados y que las modificaciones no afectaran el funcionamiento esperado.

Como resultado final, las pruebas realizadas para el caso de uso CU003 – Sumar Stock fueron satisfactorias, cumpliéndose los resultados esperados definidos para los casos de prueba.

4.4.1 Plan de pruebas CU004

El siguiente plan de pruebas básico propuesto es para el caso de uso CU004, “Crear Cliente”.

Caso de uso	Código de prueba	Tipo de prueba	Técnica propuesta	Observaciones
CU004	CPC01	Componente	Cobertura de caminos mínimos	Aplicación de caja blanca al método registrarCliente(c) de la clase ControladorCliente para asegurar que el camino del true (datos válidos) y del false (datos inválidos) respondan correctamente.
CU004	CPF02	Componente	Partición de equivalencia / Valores límite.	Aplicación de caja negra al método de autenticación ingresando datos válidos, datos inexistentes, y campos vacíos para comprobar el manejo seguro de excepciones en Java.
CU004	CPS03	Sistema	Pruebas funcionales de interfaz.	Verificación del comportamiento del flujo completo desde la pantalla gráfica. Se espera que al oprimir "Registrar" con datos correctos el cliente se registre en la base de datos y que ante un error se cancele la operación y se muestre un mensaje de alerta.

4.4.2 Modelo de pruebas

Las clases de equivalencia y valores frontera son las siguientes:

Campo: Nombre (Longitud permitida: 2 a 50 caracteres)

Equivalencias validas	
CE1	Cadena de texto con longitud entre 2 y 50 caracteres.

Equivalencias invalidas	
CE2	Campo vacío (longitud 0).
CE3	Cadena fuera de los límites permitidos (menor a 2 o mayor a 50 caracteres).

Campo: Apellido (Longitud permitida: 2 a 50 caracteres)

Equivalencias validas	
CE4	Cadena de texto con longitud entre 2 y 50 caracteres.

Equivalencias invalidas	
CE5	Campo vacío (longitud 0).
CE6	Cadena fuera de los límites permitidos (menor a 2 o mayor a 50 caracteres).

Campo: DNI (Longitud permitida: 8 caracteres)

Equivalencias validas	
CE7	Cadena de texto con longitud 8 caracteres.

Equivalencias invalidas	
CE8	Campo vacío (longitud 0).
CE9	Cadena fuera de los límites permitidos (Longitud distinta a 8).

Campo: Telefono (Longitud permitida: 8 a 20 caracteres)

Equivalencias validas	
CE10	Cadena de texto con longitud entre 8 y 20 caracteres.

Equivalencias invalidas	
CE11	Campo vacío (longitud 0).
CE12	Cadena fuera de los límites permitidos (menor a 8 o mayor a 20 caracteres).

Campo: Email (Longitud permitida: 6 a 254 caracteres)

Equivalencias validas	
CE13	Cadena de texto con longitud entre 6 y 254 caracteres.

Equivalencias invalidas	
CE14	Campo vacío (longitud 0).
CE15	Cadena fuera de los límites permitidos (menor a 6 o mayor a 254 caracteres).
CE16	Cadena que no siga el siguiente formato: *parte_local@dominio.extension*
CE17	Cadena que ya existe en la base de datos de la distribuidora.

Campo: Direccion (Longitud permitida: 6 a 50 caracteres)

Equivalencias validas	
CE18	Cadena de texto con longitud entre 6 y 50 caracteres.

Equivalencias invalidas	
CE19	Campo vacío (longitud 0).
CE20	Cadena fuera de los límites permitidos (menor a 6 o mayor a 50 caracteres).

Clase/Frontera	Datos de entrada (nombre, apellido, dni, teléfono, email, direccion)	Comportamiento esperado	Mensaje emitido por el sistema
CE1 + CE4 + CE7 + CE10 + CE13 + CE18 (Éxito)	Valentino Belli 46497315 +543413679368 valen@gmail.com Sarmiento1780	Se ingresan los datos y se registra el cliente en la base de datos.	“Cliente creado correctamente”
CE2 + CE5 + CE8 + CE11 + CE14 + CE19 (Campos vacío)	nombreCliente: [Vacío] apellidoCliente: [Vacío] dni: [Vacío] telefono: [Vacío] email: [Vacío] direccion: [Vacío]	Bloquea el envío.	“Error: Los campos no puede estar vacío."
CE3 (Frontera inferior)	nombreCliente: a	Rechazado en el cliente por el formato	“Error: El nombre del cliente debe tener entre 2 y 50 caracteres.”
CE3 (Frontera superior)	nombreCliente: aaaa

CE9 (Frontera inferior)	dni: 46597	Rechazado en el cliente por el formato	“Error: El dni del cliente debe tener 8 caracteres.”
CE9 (Frontera superior)	dni: 46497837492	Rechazado en el cliente por el formato	“Error: El dni del cliente debe tener 8 caracteres.”
CE12 (Frontera inferior)	telefono: 34136	Rechazado en el cliente por el formato	“Error: El teléfono del cliente debe tener entre 8 y 20 caracteres.”
CE12 (Frontera superior)	telefono: 341364727483729448273 23	Rechazado en el cliente por el formato	“Error: El teléfono del cliente debe tener entre 8 y 20 caracteres.”
CE15 (Frontera inferior)	email: v@g	Rechazado en el cliente por el formato	“Error: El email del cliente debe tener entre 6 y 254 caracteres.”
CE16 (Formato invalido)	email: valen@com	Rechazado en el cliente por el formato	“Error: El formato del email es incorrecto”
CE17 (Email existente)	email: valen@gmail.com	Falla la autenticación en el controlador.	"Error: El mail ya está en uso"
CE20 (Frontera inferior)	dirección: sarm1	Rechazado en el cliente por el formato	“Error: La dirección del cliente debe tener entre 6 y 50 caracteres.”

CE20 (Frontera superior)	dirección: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb 11111111111111111111	Rechazado en el cliente por el formato	“Error: La dirección del cliente debe tener entre 6 y 50 caracteres.”

Algunos de los casos de prueba para el caso de uso C004 “Crear Cliente” son los siguientes:

Nro. de caso de prueba	**Tipo de prueba**	**Descripción**	**Precondiciones**
CP-001	Funcional	Verificar que un operario pueda registrar un cliente.	El usuario operario1 tiene los permisos necesarios para registrar un cliente.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	nombreCliente: Valentino apellidoCliente: Belli dni: 46497315 telefono: +543413679368 email: valen@gmail.com direccion: Sarmiento1780	Acceder al sistema. Ingresar a la interfaz de registrar cliente. Ingresar los datos en sus respectivos campos. Presionar “Registrar”	El método retorna true, la pantalla de registrar cliente se cierra y se abre la interfaz de inicio.

Nro. de caso de prueba	Tipo de prueba	Descripción	Precondiciones
CP-002	Funcional	Verificar que el sistema deniegue el acceso ante varios campos vacíos.	El usuario operario1 tiene los permisos necesarios para registrar un cliente.
	Datos de entrada del caso de prueba	**Secuencia**	**Resultado esperado**
	nombreCliente: Valentino apellidoCliente: [Vacío] dni: 46497315 telefono: +543413679368 email: [Vacío] direccion:[Vacío]	Acceder al sistema. Ingresar a la interfaz de registrar cliente. Ingresar algunos datos en sus respectivos campos.	El sistema deshabilita el botón “Registrar” hasta que todos los campos no estén completos.

4.4.3 Tratamiento de defectos

Algunos ejemplos de posibles defectos que se podrían encontrar en un caso de uso “Crear Cliente” son estas:

Defecto ID	DEF-CU004-01
Caso de Prueba Asociado	CP-002 (Intento de registrar un cliente con datos faltantes).
Descripción del Defecto	Al registrar un cliente sin nombre, el botón “Registrar” está disponible, y al presionarlo, la base de datos cancela la operación y lanza la excepción *Error 1048: Column 'nombre' cannot be null*.
Severidad	Alta (Cancela el registro del cliente y expone datos internos del motor SQL).
Prioridad de Resolución	Alta
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación de longitud de caracteres mínimos del lado del cliente (en la pantalla) antes de invocar al controlador.

	Si no están todos los campos completos, dentro de los límites establecidos, el botón "Ingresar" no se activa y se muestra un texto de advertencia en rojo indicando que faltan campos por completar o que algunos campos están incorrectos.

Defecto ID	DEF-CU004-02
Caso de Prueba Asociado	CE3 (Nombre de cliente demasiado largo).
Descripción del Defecto	Si el operario escribe un nombre de cliente de más de 50 caracteres, la interfaz gráfica permite enviarlo, lo que provoca un colapso en la base de datos MySQL (Data truncation: Data too long for column 'nombre').
Severidad	Media (Provoca una caída controlable, pero expone datos internos del motor SQL).
Prioridad de Resolución	Media
Estado	Corregido
Tratamiento / Resolución	Se agregó una validación de longitud del lado del cliente (en la pantalla) antes de invocar al controlador. Si supera los 50 caracteres, el botón "Registrar" no se activa y se muestra un texto de advertencia en rojo.

4.4.4 Evaluación de pruebas

Las pruebas permitieron comprobar que el sistema permite registrar clientes correctamente cuando se ingresan datos válidos. También se verificó que, ante el ingreso de datos incorrectos, el sistema informa adecuadamente el error y permite al usuario volver a ingresar los datos.

Durante la ejecución de las pruebas se identificaron distintos defectos, los cuales fueron registrados y tratados. Una vez realizadas las correcciones, se repitieron las pruebas afectadas para verificar que los errores hayan sido solucionados y que las modificaciones no afectaran el funcionamiento esperado.

Como resultado final, las pruebas realizadas para el caso de uso CU004 – Registrar Cliente fueron satisfactorias, cumpliéndose los resultados esperados definidos para los casos de prueba.

5- Prototipo de interfaz

Para el caso de uso “Iniciar Sesión”, al usuario le aparecería la siguiente interfaz:



Para el caso de uso “Registrar Cliente”, al usuario le aparecería la siguiente interfaz:



Para el caso de uso “Registrar Pedido a Proveedor”, al usuario le aparecería la siguiente interfaz:



6- Definición de base de datos para el sistema

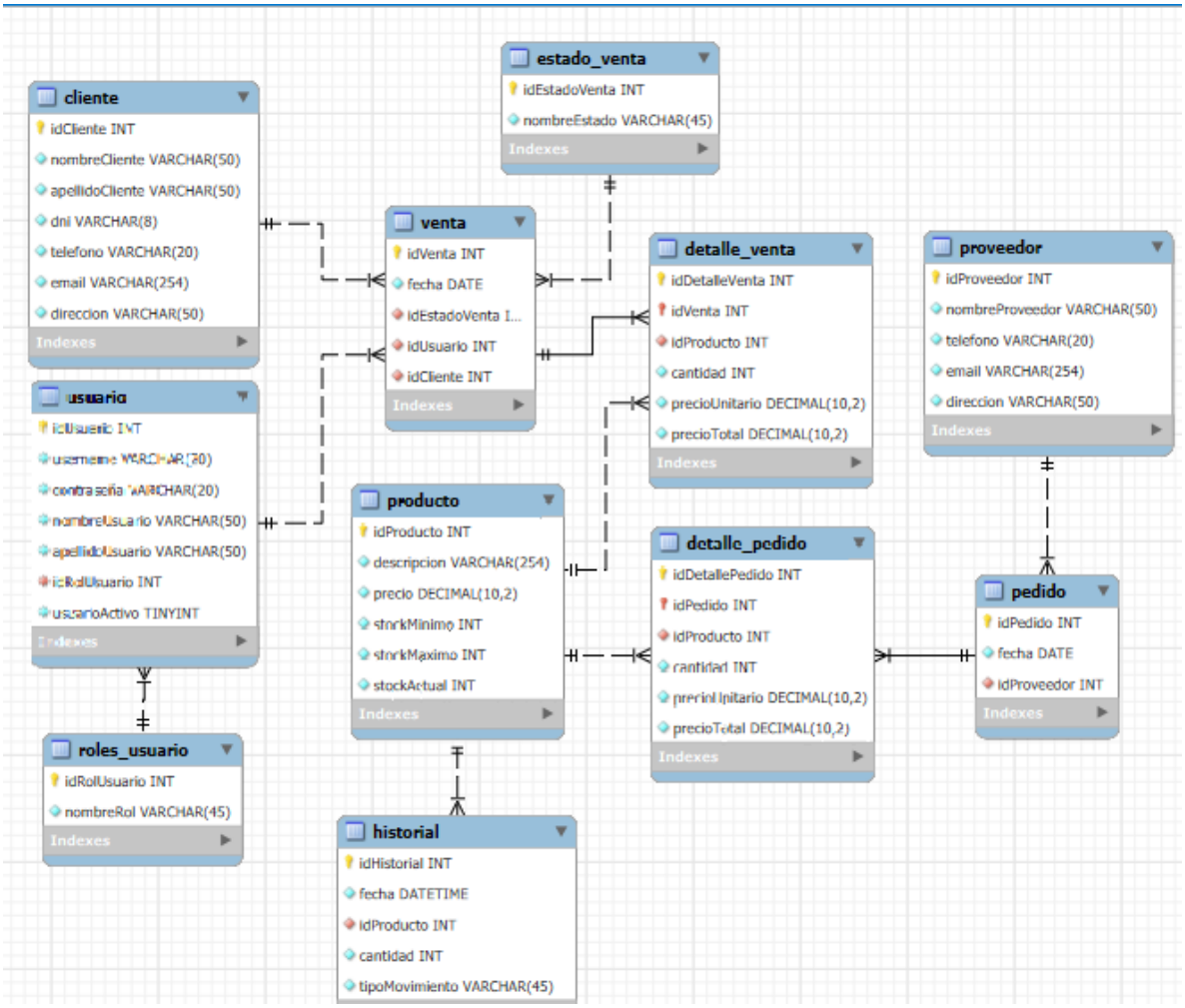
Para el desarrollo del sistema de gestión de stock, se requiere contar con una base de datos relacional. Tal cual como se especifica en los requerimientos no funcionales, en el prototipo se utiliza una base MySQL, que ofrece un buen rendimiento, flexibilidad y escalabilidad. Al considerar que se está trabajando con un prototipo, estas características son muy necesarias para avanzar de forma ordenada con la implementación del proyecto completo.

El uso de una base de datos es fundamental para lograr la persistencia de datos clave, tales como características de los productos, clientes, usuarios, entre otros. Esto es esencial para mantener un historial completo sobre todas las actividades relacionadas al movimiento del stock dentro de la empresa.

Adicionalmente, el almacenamiento de estos datos permite realizar informes en base al historial para analizar las ventas, la entrada y salida de stock, identificar tendencias de productos, etc.

7- Diagrama entidad-relación de la base de datos

El diagrama de entidad-relación (DER) de la base de datos es el siguiente:



8- Creación de las tablas MySQL

Primero creamos la base de datos:

```
-- Creacion de la base de datos
CREATE DATABASE gestion_stock_distribuidora;
USE gestion_stock_distribuidora;
```

El código para crear las tablas presentes en el diagrama entidad-relación son los siguientes.

Tabla <roles_usuario>

La tabla <roles_usuario> evita redundancias de cadenas y centraliza permisos.

```
CREATE TABLE `gestion_stock_distribuidora`.`roles_usuario` (
  `idRolUsuario` INT NOT NULL AUTO_INCREMENT,
  `nombreRol` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idRolUsuario`),
  UNIQUE INDEX `idRolUsuario_UNIQUE` (`idRolUsuario` ASC) VISIBLE);
```

Tabla <usuario>

Tabla que contiene los datos del usuario.

```
CREATE TABLE `gestion_stock_distribuidora`.`usuario` (
`idUsuario` INT NOT NULL AUTO_INCREMENT,
`username` VARCHAR(20) NOT NULL,
`contraseña` VARCHAR(20) NOT NULL,
`nombreUsuario` VARCHAR(50) NOT NULL,
`apellidoUsuario` VARCHAR(50) NOT NULL,
`idRolUsuario` INT NOT NULL,
`usuarioActivo` TINYINT NOT NULL,
PRIMARY KEY (`idUsuario`),
UNIQUE INDEX `idUsuario_UNIQUE` (`idUsuario` ASC) VISIBLE,
UNIQUE INDEX `username_UNIQUE` (`username` ASC) VISIBLE,
INDEX `fk_usuario_roles_usuario` (`idRolUsuario` ASC) VISIBLE,
CONSTRAINT `fk_usuario_roles_usuario` FOREIGN KEY (`idRolUsuario`) REFERENCES `roles_usuario` (`idRolUsuario`));
```

Tabla <cliente>

Tabla que contiene los datos del cliente

```
CREATE TABLE `gestion_stock_distribuidora`.`cliente` (
  `idCliente` INT NOT NULL AUTO_INCREMENT,
  `nombreCliente` VARCHAR(50) NOT NULL,
  `apellidoCliente` VARCHAR(50) NOT NULL,
  `dni` VARCHAR(8) NOT NULL,
  `telefono` VARCHAR(20) NOT NULL,
  `email` VARCHAR(254) NOT NULL,
  `direccion` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`idCliente`),
  UNIQUE INDEX `email_UNIQUE` (`email` ASC) VISIBLE,
  UNIQUE INDEX `idCliente_UNIQUE` (`idCliente` ASC) VISIBLE);
```

Tabla <proveedor>

Tabla que contiene los datos del proveedor

```
CREATE TABLE `gestion_stock_distribuidora`.`proveedor` (
  `idProveedor` INT NOT NULL AUTO_INCREMENT,
  `nombreProveedor` VARCHAR(50) NOT NULL,
  `telefono` VARCHAR(20) NOT NULL,
  `email` VARCHAR(254) NOT NULL,
  `direccion` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`idProveedor`),
  UNIQUE INDEX `idProveedor_UNIQUE` (`idProveedor` ASC) VISIBLE);
```

Tabla <producto>

Tabla que contiene los datos del producto

```
CREATE TABLE `gestion_stock_distribuidora`.`producto` (
  `idProducto` INT NOT NULL AUTO_INCREMENT,
  `descripcion` VARCHAR(254) NOT NULL,
  `precio` DECIMAL(10,2) NOT NULL,
  `stockMinimo` INT NOT NULL,
  `stockMaximo` INT NOT NULL,
  `stockActual` INT NOT NULL,
  PRIMARY KEY (`idProducto`),
  UNIQUE INDEX `idProducto_UNIQUE` (`idProducto` ASC) VISIBLE);
```

Tabla <pedido>

Tabla que contiene los datos básicos de un pedido

```
CREATE TABLE `gestion_stock_distribuidora`.`pedido` (
  `idPedido` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATE NOT NULL,
  `idProveedor` INT NOT NULL,
  PRIMARY KEY (`idPedido`),
  UNIQUE INDEX `idPedido_UNIQUE` (`idPedido` ASC) VISIBLE,
  INDEX `fk_pedido_proveedor` (`idProveedor` ASC) VISIBLE,
  CONSTRAINT `fk_pedido_proveedor` FOREIGN KEY (`idProveedor`) REFERENCES `proveedor` (`idProveedor`));
```

Tabla <detalle_pedido>

Tabla que contiene los detalles de un pedido

```
CREATE TABLE `gestion_stock_distribuidora`.`detalle_pedido` (
  `idDetallePedido` INT NOT NULL AUTO_INCREMENT,
  `idPedido` INT NOT NULL,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `precioUnitario` DECIMAL(10,2) NOT NULL,
  `precioTotal` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`idDetallePedido`, `idPedido`),
  INDEX `fk_detalle_pedido_pedido` (`idPedido` ASC) VISIBLE,
  INDEX `fk_detalle_pedido_producto` (`idProducto` ASC) VISIBLE,
  CONSTRAINT `fk_detalle_pedido_pedido` FOREIGN KEY (`idPedido`) REFERENCES `pedido` (`idPedido`),
  CONSTRAINT `fk_detalle_pedido_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`));
```

Tabla <historial>

Tabla que contiene los datos que se muestran en el historial

```
CREATE TABLE `gestion_stock_distribuidora`.`historial` (
  `idHistorial` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `tipoMovimiento` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idHistorial`),
  UNIQUE INDEX `idHistorial_UNIQUE` (`idHistorial` ASC) VISIBLE,
  INDEX `fk_historial_producto` (`idProducto` ASC) VISIBLE,
  CONSTRAINT `fk_historial_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`));
```

Tabla <estado_venta>

Tabla que ayuda a evitar redundancias de cadena y centralizar los posibles estados de una venta.

```
CREATE TABLE `gestion_stock_distribuidora`.`estado_venta` (
  `idEstadoVenta` INT NOT NULL AUTO_INCREMENT,
  `nombreEstado` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idEstadoVenta`),
  UNIQUE INDEX `idEstadoVenta_UNIQUE` (`idEstadoVenta` ASC) VISIBLE);
```

Tabla <venta>

Tabla que contiene los datos de una venta a cliente

```
CREATE TABLE `gestion_stock_distribuidora`.`venta` (
  `idVenta` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATE NOT NULL,
  `idEstadoVenta` INT NOT NULL,
  `idUsuario` INT NOT NULL,
  `idCliente` INT NOT NULL,
  PRIMARY KEY (`idVenta`),
  UNIQUE INDEX `idVenta_UNIQUE` (`idVenta` ASC) VISIBLE,
  INDEX `fk_venta_estado_venta` (`idEstadoVenta` ASC) VISIBLE,
  INDEX `fk_venta_usuario` (`idUsuario` ASC) VISIBLE,
  INDEX `fk_venta_cliente` (`idCliente` ASC) VISIBLE,
  CONSTRAINT `fk_venta_cliente` FOREIGN KEY (`idCliente`) REFERENCES `cliente` (`idCliente`),
  CONSTRAINT `fk_venta_estado_venta` FOREIGN KEY (`idEstadoVenta`) REFERENCES `estado_venta` (`idEstadoVenta`),
  CONSTRAINT `fk_venta_usuario` FOREIGN KEY (`idUsuario`) REFERENCES `usuario` (`idUsuario`));
```

Tabla <detalle_venta>

Tabla que contiene los detalles de una venta

```
CREATE TABLE`gestion_stock_distribuidora`.`detalle_venta` (
  `idDetalleVenta` INT NOT NULL AUTO_INCREMENT,
  `idVenta` INT NOT NULL,
  `idProducto` INT NOT NULL,
  `cantidad` INT NOT NULL,
  `precioUnitario` DECIMAL(10,2) NOT NULL,
  `precioTotal` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`idDetalleVenta`, `idVenta`),
  INDEX `fk_detalle_venta_venta` (`idVenta` ASC) VISIBLE,
  INDEX `fk_detalle_venta_producto` (`idProducto` ASC) VISIBLE,
  CONSTRAINT `fk_detalle_venta_producto` FOREIGN KEY (`idProducto`) REFERENCES `producto` (`idProducto`),
  CONSTRAINT `fk_detalle_venta_venta` FOREIGN KEY (`idVenta`) REFERENCES `venta` (`idVenta`));
```

9- Inserción, consulta y borrado de registro.

Insertar datos de prueba:

Tabla <roles_usuario>

```
INSERT INTO roles_usuario (nombreRol)
VALUES
    ('Administrador'),
    ('Encargado de almacen');
```

Tabla <usuario>

```
INSERT INTO usuario (username, contraseña, nombreUsuario, apellidoUsuario, idRolUsuario, usuarioActivo)
VALUES
    ('Admin123', 'clave123', 'Juan', 'Perez', 1, true),
    ('Pepe', 'clave222', 'Pedro', 'Diaz', 2, true),
    ('Felipe', 'clave333', 'Felipe', 'Belli', 2, true);
```

Tabla <cliente>

```
INSERT INTO cliente (nombreCliente, apellidoCliente, dni, telefono, email, direccion)
VALUES
    ('Francisco', 'Rojas', '46499888', '3413876492', 'franrj@gmail.com', 'San Martín 820, Rosario, Santa Fe'),
    ('Valentina', 'Gómez', '20948274', '3413646284', 'valen.gomez.01@yahoo.com', 'Av. Colón 2341, Rosario, Santa Fe'),
    ('Mateo', 'Fernandez', '28983250', '3413092326', 'mateo.fernandez@hotmail.com', 'Av. Pellegrini 1180, Rosario, Santa Fe');
```

Tabla <proveedor>

```
INSERT INTO proveedor (nombreProveedor, telefono, email, direccion)
VALUES
    ('Distribuidora La Cosecha S.A.', '341876543', 'contacto@lacosecha.com.ar', 'Av. Circunvalación 1250, Rosario, Santa Fe'),
    ('Alimentos del Centro S.R.L.', '341939494', 'ventas@alimentosdelcentro.com.ar', 'Av. Colón 1840, Rosario, Santa Fe'),
    ('Frigorífica y Alimentos Andinos S.A.', '341098772', 'info@alimentandinos.com.ar', 'San Martín 920, Rosario, Santa Fe');
```

Tabla <producto>

```
INSERT INTO producto (descripcion, precio, stockMinimo, stockMaximo, stockActual)
VALUES
    ('Arroz largo fino 1kg', 1500.00, 20, 100, 40),
    ('Aceite de oliva 500ml', 4500.00, 10, 60, 30),
    ('Atún en lata 170g', 2800.00, 10, 50, 20),
    ('Lentejas secas 500g', 1800.00, 15, 80, 20);
```

Tabla <pedido>

```
INSERT INTO pedido (fecha, idProveedor)
VALUES
    ('2026-09-10', 1),
    ('2026-09-10', 2),
    ('2026-09-20', 1);
```

Tabla <detalle_pedido>

```
INSERT INTO detalle_pedido (idPedido, idProducto, cantidad, precioUnitario, precioTotal)
VALUES
    (1, 1, 60, 1500, 90000.00),
    (1, 2, 30, 4500, 135000.00),

    (2, 3, 30, 2800.00, 84000.0),
    (2, 4, 40, 1800.00, 72000.00),

    (3, 4, 20, 1800.00, 36000.00);
```

Tabla <estado_venta>

```
INSERT INTO estado_venta (nombreEstado)
VALUES
    ('Pendiente'),
    ('Confirmado'),
    ('Anulado');
```

Tabla <venta>

```
INSERT INTO venta (fecha, idEstadoVenta, idUsuario, idCliente)
VALUE
    ('2026-09-05', 2, 2, 1),
    ('2026-09-19', 1, 2, 2),
    ('2026-09-15', 2, 3, 1);
```

Tabla <detalle_venta>

```
INSERT INTO detalle_venta (idVenta, idProducto, cantidad, precioUnitario, precioTotal)
VALUE
    (1, 1, 10, 1500.00, 15000.00),
    (1, 2, 1, 4500.00, 4500.00),

    (2, 4, 7, 1800.00, 12600.00),

    (3, 2, 2, 4500.00, 9000.00),
    (3, 3, 5, 2800.00, 14000.00);
```

Tabla <historial>

```sql
INSERT INTO historial (fecha, idProducto, cantidad, tipoMovimiento)
VALUES
    ('2026-09-10', 1, 60, 'Ingreso'),
    ('2026-09-10', 2, 30, 'Ingreso'),
    ('2026-09-10', 3, 30, 'Ingreso'),
    ('2026-09-10', 4, 40, 'Ingreso'),
    ('2026-09-20', 4, 20, 'Ingreso'),

    ('2026-09-05', 1, 10, 'Egreso'),
    ('2026-09-05', 2, 1, 'Egreso'),
    ('2026-09-19', 4, 7, 'Egreso'),
    ('2026-09-15', 2, 2, 'Egreso'),
    ('2026-09-15', 3, 5, 'Egreso');
```

Consulta de datos de prueba

Consultar Detalle de Ventas (Con información del Cliente y Producto)

```sql
-- Consultar Detalle de Ventas (Con información del Cliente y Producto)
SELECT
    VE.fecha AS 'Fecha Venta',
    CL.nombreCliente AS 'Nombre Cliente',
    CL.apellidoCliente AS 'Apellido Cliente',
    PR.descripcion AS 'Producto',
    DV.cantidad AS 'Cantidad',
    DV.precioUnitario AS 'Precio Unitario',
    DV.precioTotal AS 'Precio Total',
    US.username AS 'Vendedor'
FROM
    venta VE,
    cliente CL,
    detalle_venta DV,
    producto PR,
    usuario US
WHERE
    VE.idCliente = CL.idCliente
    AND VE.idVenta = DV.idVenta
    AND DV.idProducto = PR.idProducto
    AND VE.idUsuario = US.idUsuario;
```

Fecha Venta	Nombre Cliente	Apellido Cliente	Producto	Cantidad	Precio Unitario	Precio Total	Vendedor
2026-09-05	Francisco	Rojas	Arroz largo fino 1kg	10	1500.00	15000.00	Pepe
2026-09-05	Francisco	Rojas	Aceite de oliva 500ml	1	4500.00	4500.00	Pepe
2026-09-19	Valentina	Gómez	Lentejas secas 500g	7	1800.00	12600.00	Pepe
2026-09-15	Francisco	Rojas	Aceite de oliva 500ml	2	4500.00	9000.00	Felipe
2026-09-15	Francisco	Rojas	Atún en lata 170g	5	2800.00	14000.00	Felipe

Consultar Detalle de Pedidos (A Proveedores)

```sql
-- Consultar Detalle de Pedidos (A Proveedores)
SELECT
    PE.fecha AS 'Fecha Pedido',
    PV.nombreProveedor AS 'Proveedor',
    PR.descripcion AS 'Producto',
    DP.cantidad AS 'Cantidad',
    DP.precioUnitario AS 'Costo Unitario',
    DP.precioTotal AS 'Costo Total'
FROM
    pedido PE,
    proveedor PV,
    detalle_pedido DP,
    producto PR
WHERE
    PE.idProveedor = PV.idProveedor
    AND PE.idPedido = DP.idPedido
    AND DP.idProducto = PR.idProducto;
```

Fecha Pedido	Proveedor	Producto	Cantidad	Costo Unitario	Costo Total
2026-09-10	Distribuidora La Cosecha S.A.	Arroz largo fino 1kg	60	1500.00	90000.00
2026-09-10	Distribuidora La Cosecha S.A.	Aceite de oliva 500ml	30	4500.00	135000.00
2026-09-10	Alimentos del Centro S.R.L.	Atún en lata 170g	30	2800.00	84000.00
2026-09-10	Alimentos del Centro S.R.L.	Lentejas secas 500g	40	1800.00	72000.00
2026-09-20	Distribuidora La Cosecha S.A.	Lentejas secas 500g	20	1800.00	36000.00

Consultar Historial de Movimientos de Stock

```sql
-- Consultar Historial de Movimientos de Stock
SELECT
    HI.fecha AS 'Fecha Movimiento',
    PR.descripcion AS 'Producto',
    HI.cantidad AS 'Cantidad',
    HI.tipoMovimiento AS 'Tipo de Movimiento'
FROM
    historial HI,
    producto PR
WHERE
    HI.idProducto = PR.idProducto
ORDER BY
    HI.fecha DESC;
```

Fecha Movimiento	Producto	Cantidad	Tipo de Movimiento
2026-09-20	Lentejas secas 500g	20	Ingreso
2026-09-19	Lentejas secas 500g	7	Egreso
2026-09-15	Aceite de oliva 500ml	2	Egreso
2026-09-15	Atún en lata 170g	5	Egreso
2026-09-10	Arroz largo fino 1kg	60	Ingreso
2026-09-10	Aceite de oliva 500ml	30	Ingreso
2026-09-10	Atún en lata 170g	30	Ingreso
2026-09-10	Lentejas secas 500g	40	Ingreso
2026-09-05	Arroz largo fino 1kg	10	Egreso
2026-09-05	Aceite de oliva 500ml	1	Egreso

Borrado de datos de prueba

Eliminar una Venta Específica

```
-- Eliminar una Venta Especitica
-- Paso 1: Eliminar el detalle de los productos asociados a la venta
DELETE DV
FROM detalle_venta DV
WHERE DV.idVenta = 2;
-- Paso 2: Eliminar la venta principal
DELETE VE
FROM venta VE
WHERE VE.idVenta = 2;
-- Comprobacion de la eliminación de la Venta (idVenta = 2)
SELECT
    VE.idVenta AS 'ID Venta',
    VE.fecha AS 'Fecha Venta',
    DV.idProducto AS 'ID Producto',
    DV.cantidad AS 'Cantidad'
FROM
    venta VE
LEFT JOIN
    detalle_venta DV ON VE.idVenta = DV.idVenta
WHERE
    VE.idVenta = 2;
```



Borrado lógico de usuario

```
-- Borrado lógico de usuario
UPDATE usuario US
SET US.usuarioActivo = 0
WHERE US.idUsuario = 3;
-- Comprobar el estado del Usuario (Borrado Lógico)
SELECT
    US.idUsuario AS 'ID Usuario',
    US.username AS 'Nombre de Usuario',
    US.usuarioActivo AS 'Estado Activo (1=Sí, 0=No)'
FROM
    usuario US
WHERE
    US.idUsuario = 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ID Usuario	Nombre de Usuario	Estado Activo (1=Sí, 0=No)
3	Felipe	0

10- Definiciones de comunicación

Infraestructura física

En base al RNF05, toda la infraestructura informática se despliega de manera local en las oficinas centrales de la distribuidora.

- **Red de Área Local (LAN):** Se implementa una topología de red en estrella mediante un Switch Administrable de 24 puertos Gigabit Ethernet. Esto garantiza el ancho de banda necesario para soportar la concurrencia de operarios de forma simultánea sin degradar el rendimiento del sistema (RNF08).
- **Segmentación de Red**: Se configuran VLANs para aislar el tráfico de datos:
    - **VLAN Administración/Depósito**: Para las computadoras de los operarios de stock y facturistas.

    - **VLAN Servidores**: Acceso restringido únicamente para el Servidor de Aplicación y el Servidor de Base de Datos, previniendo intrusiones.
- **Hardware de los Servidores**: Servidores en formato Rack con fuentes de alimentación redundantes, conectados directamente a una UPS (Sistema de Energía Ininterrumpida) para asegurar la tolerancia a fallos y la disponibilidad 7x24 (RNF03).

Enlace de datos

Define cómo se empaquetan los bits y cómo se controla el acceso al medio físico de transmisión para evitar colisiones y pérdida de datos en el depósito.

- **Cableado Estructurado**: Se utiliza cable de red de cobre UTP Categoría 6 para todas las conexiones cableadas de las computadoras fijas, garantizando velocidades de hasta 1 Gbps y reduciendo la interferencia electromagnética de las cámaras de frío de la distribuidora.
- **Estándar IEEE 802.3 (Ethernet):** Protocolo base para la comunicación cableada en la LAN, encargado del direccionamiento físico por medio de direcciones MAC y la detección de errores mediante secuencias de verificación de tramas (FCS).

Protocolos y estándares de comunicación

El intercambio de información entre la interfaz gráfica (**JavaFX**) y el controlador se realiza de forma local en memoria mediante eventos y llamadas a métodos. Por ejemplo, para los formularios de iniciar sesión y registrar cliente, la vista captura los datos de entrada y los envía directamente al controlador interno, sin necesidad de protocolos de red ni servidores intermedios.

Para la conexión a la base de datos se utiliza el protocolo **JDBC (Java Database Connectivity)** sobre el estándar **TCP/IP** (Puerto por defecto 3306 de MySQL). Las clases DAO (Data Access Object) abren un canal de comunicación directo hacia la base de datos. Las consultas SQL (INSERT, UPDATE, SELECT) viajan

encapsuladas en tramas TCP y MySQL retorna el número de filas afectadas o conjuntos de registros como respuesta técnica.

Desarrollo TP3

El presente documento tiene como objetivo explicar el desarrollo de un sistema de control de stock destinado a una distribuidora. Se describen las tecnologías utilizadas, las decisiones tomadas durante la implementación, la organización del código, el funcionamiento general de la aplicación y los requisitos necesarios para ejecutarla correctamente.

1. Tecnologías y herramientas

La aplicación para el sistema de control de stock destinado a la distribuidora se desarrollo con las siguientes tecnologías y herramientas.

1.1. Java y Azul Zulu JDK 27

Java es el lenguaje de programación utilizado para desarrollar la lógica del sistema. El proyecto está configurado para trabajar con Azul Zulu JDK 27, una distribución del Java Development Kit (JDK).

El JDK proporciona las herramientas necesarias para compilar y ejecutar el código Java. También incluye las bibliotecas estándar de Java que permiten trabajar con clases, objetos, colecciones, archivos, excepciones y otras funcionalidades del lenguaje.

1.2. JavaFX

JavaFX es la tecnología utilizada para construir la interfaz gráfica de la aplicación.

Mediante esta biblioteca se pueden crear ventanas, formularios, botones, tablas, etiquetas, campos de texto, menús y otros componentes necesarios para que el usuario interactúe con el sistema.

Se eligió JavaFX porque permite desarrollar una aplicación de escritorio con una interfaz visual, evitando que todas las operaciones deban realizarse mediante una consola de comandos.

La interfaz está organizada en distintas vistas, cada una destinada a una funcionalidad específica. Por ejemplo, existen vistas para consultar productos, registrar nuevos productos, modificar sus datos y realizar operaciones de stock.

1.3. FXML y Scene Builder

FXML es un formato basado en XML que permite definir la estructura visual de las pantallas de JavaFX.

Se utiliza para declarar los componentes de cada vista, sus propiedades, su distribución y el controlador asociado. Esto permite separar la estructura visual de la lógica programada en Java.

Scene Builder es una herramienta visual que facilita la creación y edición de archivos FXML. Permite ubicar botones, etiquetas, tablas y campos de entrada mediante una interfaz gráfica, sin tener que escribir manualmente toda la estructura de la pantalla.

1.4. MySQL

MySQL es el sistema de gestión de bases de datos relacionales utilizado para almacenar la información de la aplicación.

La base de datos contiene las tablas necesarias para guardar los datos de las distintas entidades del sistema, como usuarios, productos y clientes, además de las que correspondan a los movimientos de stock y otras operaciones implementadas.

Se eligió una base de datos relacional porque la información del sistema está organizada en entidades que se relacionan entre sí. Esto permite consultar, insertar, modificar y eliminar registros de manera estructurada.

1.5. JDBC y MySQL Connector/J

JDBC es la tecnología de Java utilizada para establecer conexiones con bases de datos y ejecutar instrucciones SQL.

Para comunicarse con MySQL, el proyecto utiliza el controlador JDBC MySQL Connector/J, cuyo archivo se encuentra en la carpeta lib del proyecto:

lib/mysql-connector-j-26.7.0.jar

Este archivo permite que la aplicación Java se comunique con el servidor MySQL mediante JDBC. El controlador se incorpora manualmente al proyecto.

1.6. Organización de archivos

El proyecto utiliza una estructura basada en archivos Java, archivos FXML y la biblioteca JDBC necesaria para conectarse con MySQL.

Entre sus elementos principales se encuentran:

- **Archivos .java**: contienen las clases del modelo, los controladores, los DAO y la lógica de conexión.
- **Archivos .fxml**: definen la estructura visual de las pantallas.
- **Carpeta lib**: contiene el archivo JAR de MySQL Connector/J.
- **Recursos adicionales**: imágenes u otros archivos que utilice la interfaz

2. Decisiones tomadas durante el desarrollo

2.1. Separar la interfaz de la lógica del sistema

Se decidió utilizar archivos FXML para definir las pantallas y clases Java para controlar su comportamiento.

Por ejemplo, una vista puede contener un formulario para registrar un producto, mientras que su controlador se encarga de leer los campos, validar los datos y ejecutar la operación correspondiente.

Esta separación facilita los cambios visuales sin tener que modificar toda la lógica de la aplicación y permite organizar mejor el código.

2.2. Organizar el sistema por módulos

Se decidió dividir la aplicación en distintas secciones, según las operaciones que realiza cada una.

Esta organización permite trabajar sobre una funcionalidad sin tener que modificar todo el sistema. También facilita las pruebas, el mantenimiento y la incorporación de nuevas características.

Entre los módulos desarrollados o contemplados se encuentran los siguientes:

- Inicio de sesión y acceso al sistema.
- Menú principal y navegación entre pantallas.
- Gestión de usuarios.
- Gestión de productos.
- Historial de movimientos de stock.
- Gestión de clientes.
- Gestión de proveedores.
- Ventas a clientes.
- Pedidos a proveedores

2.3. Utilizar clases objeto, controlador y DAO

Se decidió representar los elementos principales del sistema mediante clases Java.

Por ejemplo, la clase *Producto* contiene los datos de un producto, mientras que la clase *Cliente* representa la información de un cliente y la clase Usuario representa a una persona que utiliza el sistema.

Los controladores administran los eventos de la interfaz, validan la información ingresada, coordinan las operaciones y muestran los mensajes correspondientes.

Las clases DAO (*Data Access Object*) se encargan de ejecutar las consultas SQL necesarias para acceder a los datos. Por ejemplo, un DAO puede insertar un cliente, buscar un producto o actualizar la información de un usuario.

Esta separación permite que las consultas a la base de datos no estén distribuidas por todos los controladores, lo que facilita su mantenimiento y reutilización.

3. Organización y funcionamiento del código

La aplicación está organizada en clases Java y archivos FXML que trabajan en conjunto.

3.1. Modelo

El modelo está formado por las clases que representan las entidades del sistema.

Algunos ejemplos son: Usuario, Producto, Cliente y MovimientoStock.

Estas clases permiten transportar y organizar la información utilizada por los controladores y los DAO.

3.2. Controladores

Los controladores conectan los componentes de las vistas con la lógica del programa.

Por ejemplo, *ProductoController* administra las acciones relacionadas con las pantallas de productos. Puede responder a los clics de los botones, leer los campos del formulario, validar los valores y solicitar las operaciones correspondientes al DAO.

3.3. DAO

Las clases DAO concentran las operaciones de acceso a la base de datos.

Algunas operaciones que pueden realizar son:

- Buscar un registro por su identificador u otro dato.
- Insertar nuevos registros.
- Actualizar registros existentes.

Por ejemplo, *ClienteDAO* contiene las consultas SQL destinadas a trabajar con los clientes. El controlador utiliza sus métodos para solicitar estas operaciones sin tener que escribir las consultas directamente en la interfaz.

3.4. Clase de conexión

La clase *ConexionMySQL* centraliza la creación de conexiones con la base de datos.

Los DAO utilizan el método de conexión para obtener un objeto Connection y ejecutar sus consultas.

Para que funcione correctamente, esta clase debe tener configurados los datos reales de conexión, como la dirección del servidor, el puerto, el nombre de la base de datos, el usuario y la contraseña.

4. Funcionamiento general de la aplicación

El funcionamiento puede describirse mediante la siguiente secuencia:

1. El usuario ejecuta la aplicación.
2. El sistema muestra la pantalla de inicio de sesión.
3. El usuario ingresa sus credenciales.
4. El sistema valida la información y determina si corresponde permitir el acceso.
5. Si la autenticación es correcta, se muestra el menú principal.
6. El usuario selecciona el módulo con el que desea trabajar.
7. La aplicación muestra la pantalla correspondiente.
8. El usuario realiza una operación, como buscar un producto, registrar un cliente o modificar información.
9. El controlador valida los datos y solicita la operación necesaria al DAO.
10. El DAO ejecuta la consulta correspondiente en MySQL.
11. El resultado vuelve al controlador, que actualiza la interfaz y muestra el mensaje correspondiente.

Si ocurre un error de validación o de acceso a los datos, el sistema debe informar la situación y evitar continuar con una operación que no se haya realizado correctamente.

5. Requisitos para ejecutar el sistema

Para ejecutar el proyecto se necesita:

- Una computadora compatible con Azul Zulu JDK 27.
- Un entorno de programación que permita configurar y ejecutar aplicaciones Java.
- Los archivos .java y .fxml del proyecto.
- La carpeta lib con el archivo mysql-connector-j-26.7.0.jar.
- Una instalación de MySQL Server.
- La base de datos creada con las tablas y columnas que utiliza el código.
- Los datos correctos de conexión configurados en la clase ConexionMySQL.
- Una instalación de JavaFX compatible con el JDK y correctamente configurada para la compilación y ejecución.

6. Pasos para preparar y ejecutar el proyecto

Paso 1. Configurar el JDK

Instalar un **Java Development Kit (JDK)** compatible con el proyecto y configurarlo en el entorno de desarrollo.

El proyecto fue desarrollado y probado utilizando Azul Zulu JDK 27. Sin embargo, no es obligatorio utilizar esa distribución específica de Java, siempre que se disponga de una versión compatible con el código y con la versión de JavaFX utilizada.

Paso 2. Configurar JavaFX

Verificar que el entorno tenga disponible JavaFX y que las bibliotecas necesarias estén configuradas para compilar y ejecutar las vistas.

Paso 3. Incorporar el controlador JDBC

Comprobar que exista la carpeta *lib* y que contenga el archivo:

mysql-connector-j-26.7.0.jar

Verificar que el JAR esté agregado a la configuración del proyecto para que las clases JDBC puedan encontrar el controlador de MySQL.

Paso 4. Preparar MySQL

Iniciar MySQL Server y crear o importar los archivos .sql de la base de datos necesaria junto con las tablas y la población de la misma.

Paso 5. Configurar la conexión

Revisar la clase *ConexionMySQL* y comprobar que los datos de conexión sean correctos: dirección del servidor, puerto, nombre de la base de datos, usuario y contraseña.

Paso 6. Compilar y ejecutar

Ejecutar el proyecto desde el IDE elegido utilizando la configuración adecuada para JavaFX y el classpath de MySQL Connector/J.

7. Conclusión

El desarrollo del sistema de control de stock permitió aplicar conceptos de programación orientada a objetos, diseño de interfaces gráficas, acceso a bases de datos relacionales y organización modular del código.

La combinación de Java, JavaFX, FXML, JDBC y MySQL proporciona una estructura adecuada para desarrollar una aplicación de escritorio que gestione información de una distribuidora.

La separación entre las vistas, los controladores, las clases del modelo y los DAO facilita la comprensión del funcionamiento del programa, permite mantener cada parte con mayor claridad y distribuir las responsabilidades para una mejor reutilización de código.

Además, el uso de una base de datos relacional permite conservar la información entre ejecuciones, mientras que las validaciones y las pruebas ayudan a detectar errores antes de que afecten a las operaciones del sistema.